

Dipartimento di Ingegneria Informatica, Modellistica, Elettronica e
Sistemistica

ROBOTICS AND AUTOMATION ENGINEERING

Modeling and control of an artificial pancreas

Dynamical Systems Theory Project

Studente:

Enrica Muto

Matricola: [281636]

Docente:

Prof. Casavola Alessandro

Indice

1	Introduction	2
2	Mathematical Model	3
2.1	State	3
2.2	Output	3
2.3	Treatment of $d(t)$	3
2.4	Parameters	4
2.5	MATLAB and Simulink implementation	4
3	Equilibrium point and linearization	6
3.1	Equilibrium point	6
3.1.1	MATLAB Implementation	7
3.2	Linearization	7
3.2.1	MATLAB implementation	8
3.2.2	Simulink implementation	8
4	System Analysis	10
4.1	Internal Stability	10
4.1.1	Modal Analysis	10
4.1.2	MATLAB implementation	10
4.2	System Evolution	11
4.2.1	Free Response Evolution	11
4.2.2	MATLAB and Simulink implementation	13
4.2.3	Forced Response Evolution	15
4.2.4	MATLAB implementation	16
4.3	External Stability	17
4.3.1	MATLAB implementation	17
5	Synthesis	19
5.1	Structural Properties	19
5.1.1	MATLAB implementation	19
5.1.2	MATLAB implementation	20
5.2	Static State Feedback	20
5.2.1	MATLAB and Simulink implementation	21
5.3	Asymptotic observer (Luenberger)	24
5.3.1	MATLAB and Simulink implementation	25
5.4	Dynamic Compensator	37
5.4.1	MATLAB and Simulink implementation	37
6	Conclusions	40

1 Introduction

Diabetes is among the ten leading causes of death affecting the adult population, representing a major problem for healthcare systems of the world.

In particular, **Diabetes Mellitus Type 1 (DMT1)** is a rapidly increasing disease. DMT1, or simply called Type 1 diabetes, is an autoimmune disease in which the immune system accidentally attacks and destroys insulin-producing β cells in the pancreas, limiting the organ's ability to produce endogenous insulin.

To manage this condition, we use a technological solution known as **artificial pancreas (AP)**. It operates as a closed-loop control system that uses a pump to administer the right amount of insulin subcutaneously based on real-time data collected by sensors, such as CGM (continuous glucose monitoring).

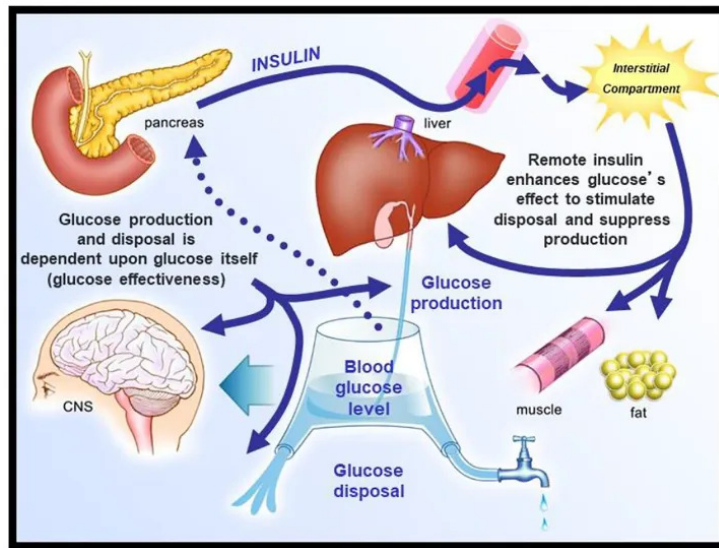


Figure 1: Closed loop system of glucose regulation.

During the last three decades, numerous control techniques have been explored to regulate an artificial pancreas in both clinical and simulated environments. To develop an effective strategy, it is important to use a reliable mathematical representation of the patient's metabolism. One of the most widely used mathematical model to describe these complex dynamics, related to the glucose-insulin regulatory system, is the **Bergman's minimal model**.

The main objective of this project is to analyze the mathematical model in order to design and implement a possible closed-loop strategy capable of providing a simple glucose-insulin monitoring and regulation method for type 1 diabetes patients. By using the tools MATLAB and Simulink it was possible to analyze the non-linear system and linearize it around an equilibrium point, and furthermore, to design and simulate control algorithms against various scenarios.

2 Mathematical Model

In this section we describe the mathematical model utilized to analyze the glucose-insulin regulatory system. While the human metabolism is highly complex, Bergman's minimal model provides a reliable and simplified representation of the overall dynamics through a set of non-linear differential equations. These equations represent three fundamental biological processes:

- Glucose absorption
- Insulin secretion
- Tissue insulin sensitivity

2.1 State

The state equations are defined as follows:

$$\begin{cases} \dot{G}(t) = -(X(t) + p_1)G(t) + p_1G_b + d(t) \\ \dot{X}(t) = -p_2X(t) + p_3(I(t) - I_b) \\ \dot{I}(t) = -p_4(I(t) - I_b) + u(t) \end{cases} \quad (1)$$

where:

- $G(t)$ is the plasma glucose level
- $X(t)$ is the insulin action
- $I(t)$ is the plasma insulin level
- $u(t)$ is the control input, representing the exogenous insulin that the AP infuses subcutaneously and enters in the bloodstream.
- $d(t)$ is an external input representing the meal. In other words it's the glucose absorption rate into the blood via food.

As we can see from the system (1), the non-linear term is contained in the first equation: $X(t) \cdot G(t)$.

2.2 Output

For the analysis presented in the following chapters, it is essential to define the output of the system. Generally, the only measurable state is the plasma glucose via a sensor called Continuous Glucose Monitor (CGM). Therefore, the system's output equation is defined as:

$$y(t) = G(t) \quad (2)$$

2.3 Treatment of $d(t)$

The model includes a non-manipulable input $d(t)$. This type of input usually can be approximated using complex mathematical models, such as the Fisher meal model. However, for this study, we have preferred to consider the input $d(t)$ as an ideal pulse in order to simplify the overall analysis.

It's crucial to clarify how this input is handled across the different phases of the project. During the analysis of the non-linear plant (i.e. evaluation around the equilibrium point) the input was considered to show its impact on the system's dynamics. On the contrary, during the synthesis phase, $d(t)$ is deliberately excluded from the design for two specific reasons:

- **Controller Design:** since $d(t)$ is a non-manipulable input, it cannot be governed by a control law. The controller (SSF) is solely designed to counteract the effect of $d(t)$ by manipulating the input $u(t)$.
- **Observer Design:** here $d(t)$ is treated as an unknown input. In the real world, there is no sensor capable to measure the exact meal absorption rate. Generally, the patient manually enter the carbohydrate intake, eventually introducing inaccuracies.

By deliberately hiding $d(t)$ from the synthesis phase, we mimic the unmeasured and non-manipulable nature of real-world scenarios.

2.4 Parameters

The parameters used for the simulation of the model are:

Parameter	Symbol	Value	Units
Glucose base level before injection	G_b	119.1858	$[mg/dL]$
Insulin base level before injection	I_b	15.3872	$[\mu U/ml]$
Insulin independent constant	p_1	0.0287	$[1/min]$
Decrease rate of tissue's glucose uptake	p_2	0.0283	$[1/min]$
Enhanced glucose uptake capability	p_3	$5.035 \cdot 10^{-5}$	$[L/(\mu U)]$
Insulin degradation rate	p_4	5/54	$[1/min]$

Tabella 1: Parameters of Bergman's Minimal Model.

2.5 MATLAB and Simulink implementation

To implement the model in MATLAB, we firstly define the parameters:

```

1 %Type 1 Diabetes Model Parameters
2 p1 = 0.0287;
3 p2 = 0.0283;
4 p3 = 5.035*10^(-5);
5 p4 = 5/54;
6
7 Gb = 119.1858;
8 Ib = 15.3872;

```

Listing 1: Linearization around the Equilibrium point using MATLAB.

These variables are then passed to Simulink, where the differential equations (1) are constructed using blocks such as Gain, Integrators and Sums.

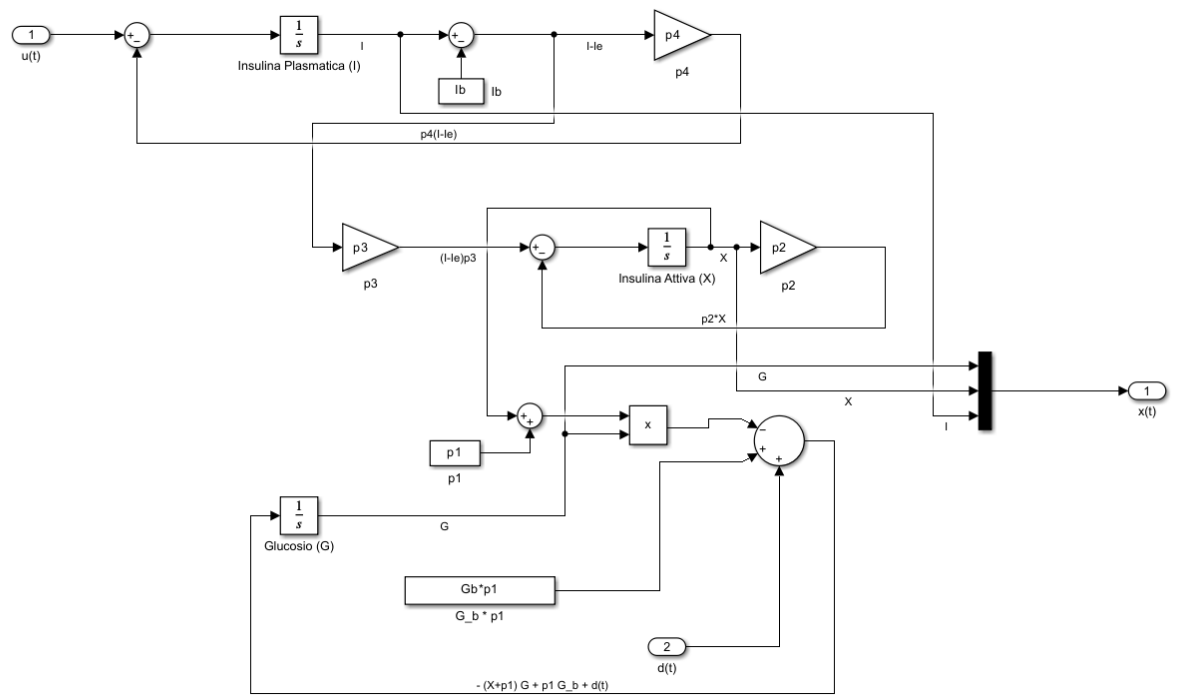


Figura 2: Plant of the system on Simulink.

3 Equilibrium point and linearization

We are now interested in analyzing certain properties of the system using concepts from linear systems theory. However, our system is nonlinear. In general, such a system can be described using a Locally Representation I/S/O (Input/State/Output):

$$\begin{cases} \dot{x}(t) = f(x(t), u(t)) \\ y(t) = \eta(x(t), u(t)) \end{cases} \quad (3)$$

Where our state vector $x(t)$ is defined as a vector of dimension 3:

$$\dot{\mathbf{x}}(\mathbf{t}) = \begin{bmatrix} \dot{x}_1(t) = \dot{G}(t) \\ \dot{x}_2(t) = \dot{X}(t) \\ \dot{x}_3(t) = \dot{I}(t) \end{bmatrix} \quad (4)$$

To linearize the system, the first step is to compute an equilibrium point. This point will serve as our nominal operating condition, around which the linear approximation is computed and considered valid.

Here, we introduce the concept of nominal solution and perturbed solution. Let the nominal solution be:

$$\hat{x}_0, \hat{u}(t) \rightarrow \hat{x}(t), \hat{y}(t) \quad (5)$$

This represents the value of the function in the equilibrium point, the system's ideal trajectory. Meanwhile the perturbed solution arise when the initial conditions deviate from the nominal ones, such that $u(t) \neq \hat{u}(t)$ and $x(t) \neq \hat{x}(t)$.

$$x_0, u(t) \rightarrow x(t), y(t) \quad (6)$$

The error (or perturbations) is defined as the difference between the perturbed solution and the nominal ones:

$$\tilde{x} = x(t) - \hat{x}(t) \quad (7)$$

By expressing the system dynamics in terms of these perturbations, we can perform a first-order Taylor expansion around the nominal solution:

$$f(\hat{x}(t) + \tilde{x}, \hat{u} + \tilde{u}) = f(\hat{x}, \hat{u}) + \left. \frac{\partial f}{\partial x} \right|_{(\hat{x}, \hat{u})} \tilde{x}(t) + \left. \frac{\partial f}{\partial u} \right|_{(\hat{x}, \hat{u})} \tilde{u}(t) + o(\tilde{x}, \tilde{u}) \quad (8)$$

Where $o(\tilde{x}, \tilde{u})$ is the approximation error of higher order terms. The same consideration can be made for the output $y(t)$.

3.1 Equilibrium point

A non-linear system can have multiple equilibrium points. In order to compute the equilibrium point for our model it is necessary to assume the absence of external disturbances ($d(t) = 0$) and control inputs ($u(t) = 0$).

Why do we consider $d(t) = 0$ and $u(t) = 0$? We are interested in compute the nominal equilibrium point of the system.

$$\dot{x}(t) = f(x_{eq}, \bar{u}) = 0, \quad \text{with } \bar{u} \text{ a constant value and } x_{eq} \text{ an equilibrium point} \quad (9)$$

In reality, this corresponds to a steady state where the patient is fasting (no glucose intake from meals, $d(t) = 0$) and no exogenous insulin is being administered by the artificial pancreas ($u(t) = 0$).

Under these steady-state conditions, the state derivatives are set to zero ($\dot{x}(t) = 0$), allowing us to find the basal values of the state variables:

$$\dot{x}_3(t) = 0 \rightarrow -p_4(x_3(t) - I_b) + u(t) = 0 \rightarrow x_{3,eq} = I_b \quad (10)$$

$$\dot{x}_2(t) = 0 \rightarrow -p_2x_2(t) + p_3(x_3(t) - I_b) = 0 \rightarrow x_{2,eq} = 0 \quad (11)$$

$$\dot{x}_1(t) = 0 \rightarrow -(x_2(t) + p_1)x_1(t) + p_1G_b + d(t) = 0 \rightarrow x_{1,eq} = G_b \quad (12)$$

Therefore, our equilibrium point is:

$$x_{eq} = \begin{bmatrix} x_{1,eq} \\ x_{2,eq} \\ x_{3,eq} \end{bmatrix} = \begin{bmatrix} G_b \\ 0 \\ I_b \end{bmatrix} = \begin{bmatrix} 119.1858 \\ 0 \\ 15.3872 \end{bmatrix} \quad (13)$$

3.1.1 MATLAB Implementation

Alternatively, this process can be simply done by using the MATLAB command `trim` function. By defining the initial guesses and specifying which variables must be held constant,, we can find our equilibrium point from the simulink model, already discussed before.

```
1 % Initial conditions
2 x0 = [Gb; 0; Ib];
3 u0 = [0; 0];
4 y0 = x0;
5 idu = [1; 2];
6
7 %Equilibrium point
8 [xeq, ueq] = trim('Pianta', x0, u0, y0, [], idu, []);
```

Listing 2: Equilibrium point computation using MATLAB.

As we can see, running this script yields the same theoretical equilibrium state calculated analytically.

```
1 xeq =
2
3     119.1858
4         0
5     15.3872
6
7
8 ueq =
9
10        0
11        0
```

Listing 3: Equilibrium point computation using MATLAB.

3.2 Linearization

Once the equilibrium point has been computed, we can proceed to linearize the system around it. To do so, we use evaluate Jacobian matrices evaluated at the nominal solution:

$$\begin{aligned} A &= \left. \frac{\partial f(x, u)}{\partial x} \right|_{(\hat{x}, \hat{u})} & B &= \left. \frac{\partial f(x, u)}{\partial u} \right|_{(\hat{x}, \hat{u})} \\ C &= \left. \frac{\partial \eta(x, u)}{\partial x} \right|_{(\hat{x}, \hat{u})} & D &= \left. \frac{\partial \eta(x, u)}{\partial u} \right|_{(\hat{x}, \hat{u})} \end{aligned} \quad (14)$$

And we obtain the following linear model:

$$\begin{cases} \dot{\tilde{x}}(t) = A\tilde{x}(t) + B\tilde{u}(t) \\ \tilde{y}(t) = C\tilde{x}(t) + D\tilde{u}(t) \\ \tilde{x}(0) = \tilde{x}_0 \end{cases} \quad (15)$$

where:

$$\tilde{x}(t) = x(t) - x_{eq}\tilde{u}(t) = u(t) - u_{eq}\tilde{y}(t) = y(t) - y_{eq} \quad (16)$$

3.2.1 MATLAB implementation

To compute the linearization in MATLAB, we use the command `linmod`.

```
1 %linearization of the model
2 [A,B,C,D] = linmod('Pianta', xeq,ueq)
3
4 %State Space Representation
5 sys = ss(A,B,C,D);
```

Listing 4: Linearization around the Equilibrium point using MATLAB.

The results:

```
1 A =
2
3     -0.0287    -119.1858         0
4         0     -0.0283     0.0001
5         0         0    -0.0926
6
7
8 B =
9
10        0         1
11        0         0
12        1         0
13
14
15 C =
16
17        1         0         0
18
19
20 D =
21
22        0         0
```

Listing 5: Linearization around the Equilibrium point using MATLAB.

We can notice that D is a zero matrix. From theory, we know that when matrix D is equal to zero, the input-output transfer function is strictly proper (i.e. strictly causal).

3.2.2 Simulink implementation

We can implement the linear model in Simulink:

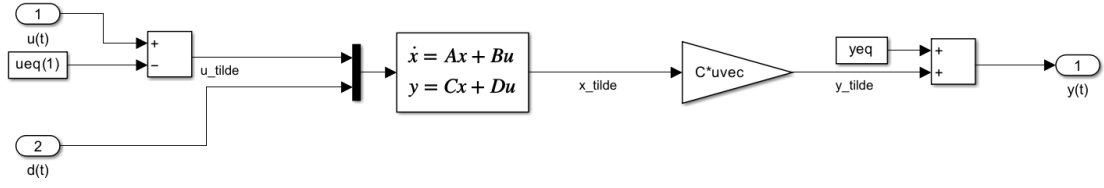


Figura 3: Linearized model around the equilibrium point.

The Bergman minimal model used in this study represents a MISO (Multiple-Input Single-Output) system, as it is driven by two independent external inputs and use a single measured output.

4 System Analysis

4.1 Internal Stability

Stability is one of the most important properties of a system. It is categorized into two classes: internal stability and external stability. We focus on the first category.

Internal stability is related to the study of the stability of the nominal solution with respect to perturbations of the nominal initial state (generally, $\hat{u}(t) = 0$).

4.1.1 Modal Analysis

Consider the system

$$\dot{x}(t) = f(x(t)) \quad (17)$$

We are interested in analyzing the stability of the equilibrium point x_{eq} , which represents the nominal solution of interest.

By investigating the eigenvalues of the system matrix A , we can determine whether the system is stable or not.

What is an eigenvalue? Let $A \in \mathbb{R}^{n \times n}$ be a square matrix, and let $v \in \mathbb{C}^n$ be a non-zero vector such that

$$Av = \lambda v$$

for some scalar λ . Then λ is called an eigenvalue of the matrix A and v is called an eigenvector of A associated with λ .

These eigenvalues can be computed by solving the characteristic polynomial $\det(A - \lambda I) = 0$.

4.1.2 MATLAB implementation

Alternatively, we can use the command `eig` in MATLAB.

```
1 eigenvalues = eig(A)
```

Listing 6: Eigenvalues using MATLAB.

```
1 eigenvalues =  
2  
3 -0.0287  
4 -0.0283  
5 -0.0926
```

Listing 7: Eigenvalues using MATLAB.

We can observe that all three eigenvalues are real and negative, which indicates that the system is asymptotically stable. The reason behind this stability, is given by the Reduced Lyapunov criterion.

Reduced Lyapunov Criterion Given $\dot{x}(t) = Ax(t)$ where $A = \left. \frac{\partial f(x)}{\partial x} \right|_{x=x_{eq}}$ is the Jacobian matrix obtained by linearizing the system around the equilibrium point x_{eq} , then stability of this equilibrium point is classified as:

1. Locally asymptotically stable if all eigenvalues of A satisfy $\text{Re}(\lambda_i) < 0, \forall \lambda_i \in \text{sp}(A)$
2. Unstable if A has at least one eigenvalue with $\text{Re}(\lambda_i) > 0$
3. No conclusion can be drawn on the non-linear system in all other cases.

As we can see, the property of stability of the computed equilibrium point perfectly matches the reality: a fasting person's metabolism is stable, even under small internal perturbations, the glucose concentration returns to its basal level.

4.2 System Evolution

The linearized continuous-time system (local I/S/O representation) under consideration, is the following:

$$\begin{cases} \dot{\tilde{x}}(t) = A\tilde{x}(t) + B\tilde{u}(t) \\ \tilde{y}(t) = C\tilde{x}(t) \\ \tilde{x}(0) = \tilde{x}_0 \end{cases} \quad (18)$$

where the state-space matrices are:

$$A = \begin{bmatrix} -0.0287 & -119.1858 & 0 \\ 0 & -0.0283 & 0.0001 \\ 0 & 0 & -0.0926 \end{bmatrix} \quad B = \begin{bmatrix} 0 & 1 \\ 0 & 0 \\ 1 & 0 \end{bmatrix} \quad (19)$$

$$C = [1 \quad 0 \quad 0] \quad D = [0 \quad 0] \quad (20)$$

Now we want to consider the global representation I/S/O in order to evaluate the response of the system over time (i.e analyzing the free response and the forced response). To do so, we use the Lagrange formula to compute the explicit solution for the state $x(t)$ and for the output $y(t)$.

$$\begin{cases} \dot{\tilde{x}}(t) = \underbrace{e^{At}\tilde{x}_0}_{\text{Free Response}} + \underbrace{\int_0^t e^{A(t-\tau)}B\tilde{u}(\tau)d\tau}_{\text{Forced Response}} \\ \tilde{y}(t) = \underbrace{Ce^{At}\tilde{x}_0}_{\text{Free Response}} + \underbrace{\int_0^t Ce^{A(t-\tau)}B\tilde{u}(\tau)d\tau + D\tilde{u}(t)}_{\text{Forced Response}} \\ \tilde{x}(0) = \tilde{x}_0 \end{cases} \quad (21)$$

The linearized system is a linear time-invariant (LTI) system, it respects the principle of superposition. This means that we can separate the response into two distinct contributions:

- free response: it describes how the state and output evolves solely due to the initial condition, assuming no external input is applied
- forced response: it describes how the state and output evolves solely due to the applied input, assuming zero initial conditions

4.2.1 Free Response Evolution

As we can see the free response is linked to the study of the exponential matrix and initial condition:

$$\begin{cases} \tilde{x}(t) = e^{At}\tilde{x}_0 \\ \tilde{y}(t) = Ce^{At}\tilde{x}_0 \\ \tilde{x}(0) = \tilde{x}_0 \end{cases} \quad (22)$$

The nature of matrix A is linked to its eigenvalue and eigenvector. The eigenvalues (previously calculated) are:

$$\lambda_1 = -0.0287 \quad \lambda_2 = -0.0283 \quad \lambda_3 = -0.0926 \quad (23)$$

Meanwhile the eigenvectors related to the eigenvalue $\lambda_i, i = 1, 2, 3$, can be computed in MATLAB by using the following procedure:

```
1 [V,D,W] = eig(A)
```

Listing 8: Eigenvalues and eigenvectors using MATLAB.

where D is a diagonal matrix of eigenvalues, V is a matrix whose columns are the corresponding right eigenvectors and W is a matrix whose columns corresponds to the left eigenvectors:

```
1 V =
2
3     1.0000    -1.0000    -0.8252
4         0     0.0000    -0.0004
5         0         0     0.5649
6
7
8 D =
9
10    -0.0287         0         0
11         0    -0.0283         0
12         0         0    -0.0926
13
14
15 W =
16
17     0.0000         0         0
18     1.0000     1.0000         0
19     0.0008     0.0008     1.0000
```

Listing 9: Eigenvalues and eigenvectors using MATLAB.

From theory, we know that the state $\tilde{x}(t)$ can be rewritten by using its spectral decomposition formula:

$$\tilde{x}(t) = \sum_{i=1}^n (w_i^T \tilde{x}_0) e^{\lambda_i t} v_i \quad (24)$$

where:

- $e^{\lambda_i t}$ is the system's natural mode, related to the eigenvalue λ_i . It defines the temporal evolution (i.e. how fast the state converges or diverges)
- v_i is the right eigenvector related to the eigenvalue λ_i . It defines the direction in the state space
- w_i^T is the left eigenvector related to the eigenvalue λ_i . It defines the weight that influence the initial condition $\tilde{x}_0$

Given an initial condition $x_0 = [x_{10}, x_{20}, x_{30}]^T$, and by using the previous formula, we can write the explicit solution as:

$$\tilde{x}(t) = (w_1^T \tilde{x}_0) e^{(-0.0287)t} v_1 + (w_2^T \tilde{x}_0) e^{(-0.0283)t} v_2 + (w_3^T \tilde{x}_0) e^{(-0.0926)t} v_3 \quad (25)$$

In this particular case, all the eigenvalues λ_i are real, negative and distinct numbers. This means that firstly the modes converge (i.e. $\lim_{t \rightarrow \infty} e^{\lambda_i t} = 0, i = 1, 2, 3$), and most importantly since the modes are real, they will never oscillate.

The dominant eigenvalue is $\lambda_2 = -0.0283$. Correspondingly, v_2 and w_2 are the dominant right and left eigenvectors, respectively. This eigenvalue corresponds to the slowest natural mode and it is the one that will have an impact on the overall settling time of the system.

Furthermore, the convergence speed of each mode can be calculated as the following time constant:

$$\tau_{time} = -\frac{1}{\lambda} \quad (26)$$

in our case for

- Mode 1: $\tau_{time} = 34.84$ minutes
- Mode 2: $\tau_{time} = 35.34$ minutes
- Mode 3: $\tau_{time} = 10.80$ minutes

4.2.2 MATLAB and Simulink implementation

We can now proceed to simulate the behaviour of the free response in Simulink and then plot it in MATLAB.

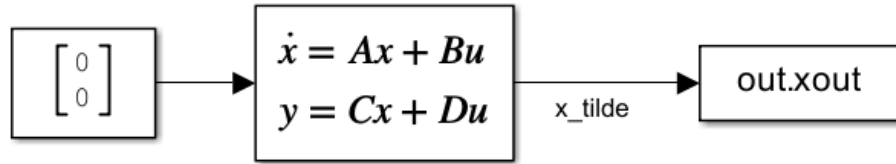


Figura 4: Linear model for the free response.

Two cases are considered, by using a simulation of 500(min):

- **Case 1:** $\tilde{x}_0 = [50, 0, 0]^T$ corresponding to a hyperglycaemia case
- **Case 2:** $\tilde{x}_0 = [50, 0, 20]^T$ corresponding to a hyperglycaemia case with residual plasma insulin

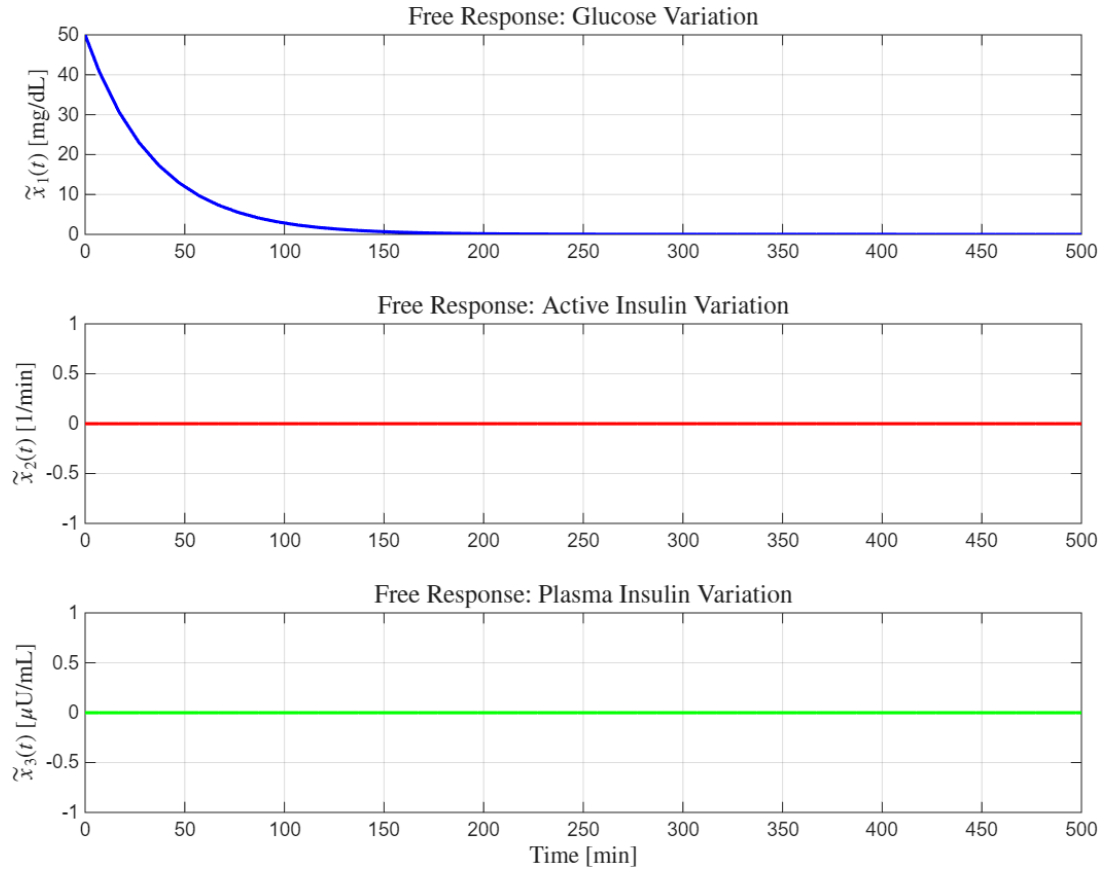


Figura 5: Free evolution for each state component, with $\tilde{x}_0 = [50, 0, 0]^T$

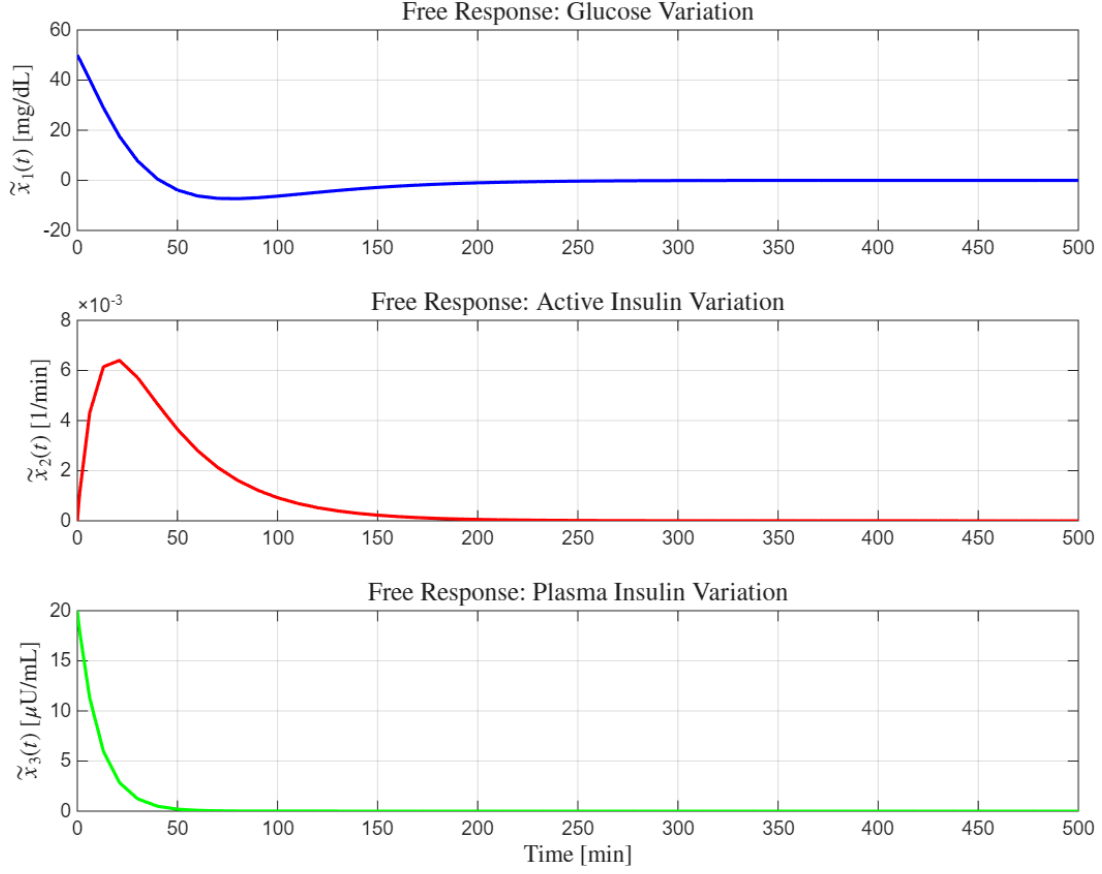


Figure 6: Free evolution for each state component, with $\tilde{x}_0 = [50, 0, 20]^T$

As we can observe, the simulated behaviour of the free responses strictly follow the theoretical aspects previously discussed: every state component converges to zero, even if the convergence rate is really slow.

4.2.3 Forced Response Evolution

It's not always so simple to compute a solution in the time domain for the forced response.

$$\begin{cases} \tilde{x}(t) = \int_0^t e^{A(t-\tau)} B \tilde{u}(\tau) d\tau \\ \tilde{y}(t) = \int_0^t C e^{A(t-\tau)} B \tilde{u}(\tau) d\tau + D \tilde{u}(t) \end{cases} \quad (27)$$

That is why it is approached in the transformed domain, using the Laplace's Transform:

$$\begin{cases} \tilde{X}(s) = W_{\tilde{x}, \tilde{u}}(s) \tilde{U}(s) \\ \tilde{Y}(s) = W_{\tilde{y}, \tilde{u}}(s) \tilde{U}(s) \end{cases} \quad (28)$$

where:

- $\tilde{X}(s) = \mathcal{L}\{\tilde{x}(t)\}$
- $\tilde{Y}(s) = \mathcal{L}\{\tilde{y}(t)\}$
- $\tilde{U}(s) = \mathcal{L}\{\tilde{u}(t)\}$
- $W_{\tilde{x}, \tilde{u}} = (sI - A)^{-1}B$
- $W_{\tilde{y}, \tilde{u}} = C(sI - A)^{-1}B + D$

4.2.4 MATLAB implementation

Since the linearized model is a MISO LTI system, we can use the principle of superposition to analyze the forced response by decoupling the two inputs, insulin pump $u(t)$ and meal disturbance $d(t)$. To compute the transfer function we can separate the B and D matrices and compute each function by using the command `tf`. Alternatively, we can compute numerator and denominator for each inputs via the `ss2tf` command and then compute `tf(num,den)`.

```
1 %Separate the two inputs
2 Bu = [0; 0; 1];
3 Bd = [1; 0; 0];
4
5 %State-Space for each inputs
6 sys_u = ss(A, Bu, C, D(1,1));
7 sys_d = ss(A, Bd, C, D(1,1));
8
9 %Transfer Function
10 Wu = tf(sys_u);
11 Wd = tf(sys_d);
```

Listing 10: Transfer Function computed using MATLAB.

The transfer functions related to the inputs $u(t)$ and $d(t)$ are respectively:

```
1 Wu =
2
3          -0.006001
4  -----
5  s^3 + 0.1496 s^2 + 0.00609 s + 7.52e-05
6
7
8 Wd =
9
10         1
11  -----
12  s + 0.0287
```

Listing 11: Transfer Function computed using MATLAB.

Now we proceed to simulate the system's forced response over a time of 300(min). To evaluate the system behaviour, we apply a unitary step defined as:

$$\tilde{u}(t) = \begin{cases} 1, & t \geq 0 \\ 0, & t < 0 \end{cases} \quad (29)$$

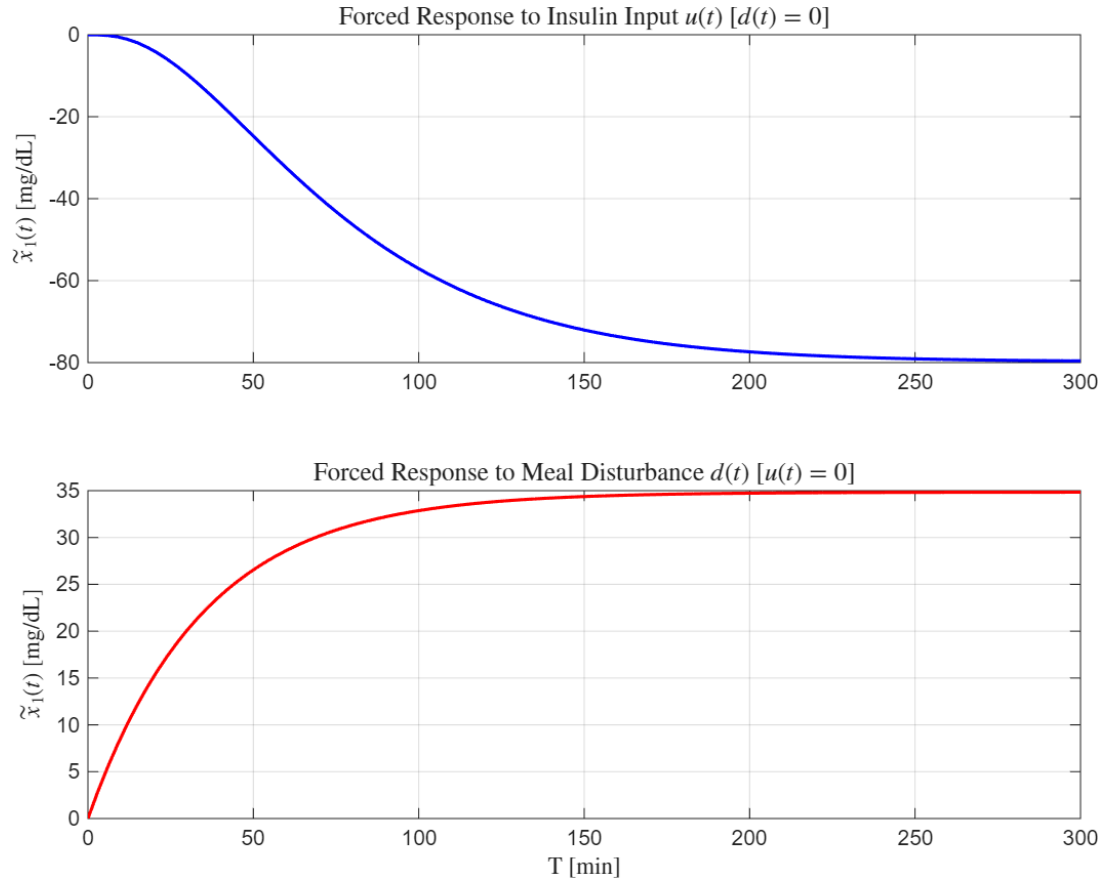


Figure 7: Forced response of state $\tilde{x}_1(t)$.

As we can observe from the plot, the system behaves exactly as the real dynamics:

- Response to insulin: a unit step of insulin produce a reduction in blood glucose levels (hypoglycaemia).
- Response to meal disturbance: a unit step in the meal disturbance cause an increase in blood glucose levels (hyperglycaemia).

4.3 External Stability

External stability is related to the study of stability of the nominal solution with respect to perturbations of the nominal input. We focus on the analysis of the transfer function of the system, in this case $W_{\tilde{y}\tilde{u}}$ and $W_{\tilde{y}\tilde{d}}$. In particular, the BIBO (Bounded-Input Bounded-Output) stability theorem states that the continuous time system is BIBO stable if all poles of the transfer function are asymptotically stable (i.e. $\text{Re}(p_i) < 0$).

4.3.1 MATLAB implementation

This property can be easily verified on MATLAB by using the command `pole` for both the transfer functions:

```
1 %Compute poles
2 Pu = pole(Wu)
3 Pd = pole(Wd)
```

Listing 12: Poles computed using MATLAB.

which results in:

```

1 Pu =
2
3     -0.0926
4     -0.0287
5     -0.0283
6
7
8 Pd =
9     -0.0287

```

Listing 13: Poles computed using MATLAB.

The system is BIBO stable. We can observe that, according to the theory, the poles are a subset of the eigenvalues of the state matrix A . Furthermore, $W_{\tilde{y}\tilde{u}}$ is linked to all the three eigenvalues, meanwhile $W_{\tilde{y}\tilde{d}}$ has a single eigenvalue related to the glucose component $G(t)$.

5 Synthesis

We are now interested in synthesizing a controller capable of improving the convergence rate and the overall stability of the system. In particular, the controller is designed in order to manage the control input related to the insulin pump, while we will treat the external input related to the meal as a non-manipulable variable.

5.1 Structural Properties

Structural properties describe how a system react to inputs (i.e. input-state bonding: are the actuators sufficient and correctly positioned?) or how the outputs are influenced by the internal state of the system (i.e. output-state bonding: are the sensors sufficient and correctly positioned?). The most important properties are:

- Reachability (input-state bonding)
- Controllability (input-state bonding)
- Observability (output-state bonding)
- Reconstructability (output-state bonding)

The two properties that we are interested in analyzing for our control design are reachability and observability.

Reachability Starting from an initial state $x_0 = x_{eq}$, a state x_f is said to be reachable if there exist a control input $u(t)$ that drives the state $x(t)$ to x_f within a finite time interval t . To verify if a LTI system is reachable or not, we need to analyze the reachability matrix, defined as:

$$\mathcal{R} = [B \quad AB \quad A^2B \quad \dots \quad A^{n-1}B] \in \mathbb{R}^{n \times nm} \quad (30)$$

Therefore, the Reachability Condition states that the matrix $\mathcal{R}$ must be full rank (i.e $\text{rank}(\mathcal{R}) = n$, where n is the dimension of the state vector). Since we are studying a continuous-time LTI system, the concept of controllability and reachability are mathematically the same.

5.1.1 MATLAB implementation

We can compute the reachability matrix by using the command `ctrb`.

```
1 R = ctrb(A,Bu)
2 rank_R = rank(R)
```

Listing 14: Reachability using MATLAB.

Obtaining the following results:

```
1 R =
2
3      0      0   -0.0060
4      0   0.0001  -0.0000
5   1.0000  -0.0926   0.0086
6
7
8 rank_R =
9
10      3
```

Listing 15: Reachability using MATLAB.

As we can see the Reachability condition is met.

Observability A system is said to be observable if given the knowledge of the input and the output, it is possible to determine any unknown initial state x_0 within a finite interval $[0, T]$. Similarly to reachability, this is verified by checking if the observability matrix $\mathcal{O}$ is full rank. This matrix is defined as:

$$\mathcal{O} = \begin{bmatrix} C \\ CA \\ CA^2 \\ \vdots \\ CA^{n-1} \end{bmatrix} \in \mathbb{R}^{np \times n} \quad (31)$$

5.1.2 MATLAB implementation

We can compute the observability matrix by using the command `obsv`.

```
1 O = obsv(A,C)
2 RankO = rank(O)
```

Listing 16: Observability using MATLAB.

Obtaining the following results:

```
1 O =
2
3     1.0000         0         0
4    -0.0287   -119.1858         0
5     0.0008     6.7936    -0.0060
6
7
8 RankO =
9
10      3
```

Listing 17: Observability using MATLAB.

Since the condition is met, we can say that the system is completely observable.

5.2 Static State Feedback

As we discussed before, we are interest in improving the convergence rate of the system toward the fasting equilibrium point. From a practical perspective, a slow convergence to the equilibrium could lead to life-threatening situation for the patient, such as prolonged periods of hyperglycaemia or hypoglycaemia.

A common solution is to implement a control law. One of the simplest feedback control strategies is the Static State Feedback (SSF). Let's consider the following linearized open-loop system:

$$\begin{cases} \dot{\tilde{x}}(t) = A\tilde{x}(t) + B\tilde{u}(t) \\ \tilde{y}(t) = C\tilde{x}(t) + D\tilde{u}(t) \end{cases} \quad (32)$$

Choosing as input (i.e. control law):

$$\tilde{u}(t) = K\tilde{x}(t) \quad (33)$$

where:

$$K = [K_1 \ K_2 \ \dots \ K_n] \in \mathbb{R}^{m \times n} \quad (34)$$

Substituting the control law (33) into the system equation (32), we obtain the following closed-loop system:

$$\begin{cases} \dot{\tilde{x}}(t) = (A + BK)\tilde{x}(t) \\ \tilde{y}(t) = (C + DK)\tilde{x}(t) \end{cases} \quad (35)$$

Since the system is fully reachable we can arbitrarily allocate the eigenvalues to the desired position. In this case, we select poles such that will improve the system, by generating a faster transient response while preserving closed-loop stability.

It's important to emphasize that the Static State Feedback (SSF) designed here represent an ideal control strategy. The design assumes that all the state variables are known (measurable in real time), which is not the case in a real Artificial Pancreas (typically, we measure only the blood glucose level $y(t) = G(t)$).

5.2.1 MATLAB and Simulink implementation

For the following implementation, we consider the non-linearized system model and apply the control law previously defined (33). To compute the gain matrix K in MATLAB, we can use either the Ackerman's formula (note that for this procedure in Matlab, the `acker` command accept only systems with single-input) or the command `place` (which relies on a more robust algorithm and supports multi-input systems).

It's important to check the eigenvalues of $(A + BK)$ to ensure that they are all asymptotically stable. This condition is respected since the open loop system (A, B) is completely reachable.

```
1 %Choose closed-loop poles
2 poles_SSF = [-0.2, -0.3, -0.4];
3
4 %Compute the state feedback gain matrix
5 K = place(A, B_u, poles_SSF);
```

Listing 18: Static State Feedback design using MATLAB.

Resulting in:

```
1 K =
2
3 -0.0079    88.7148    0.0604
```

Listing 19: Static State Feedback design using MATLAB.

Here the Simulink implementation:

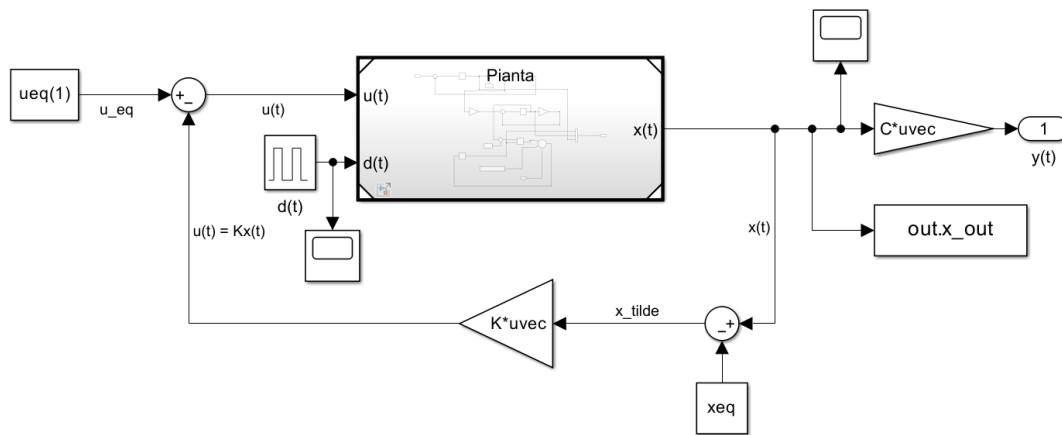


Figura 8: Closed-loop system using Simulink.

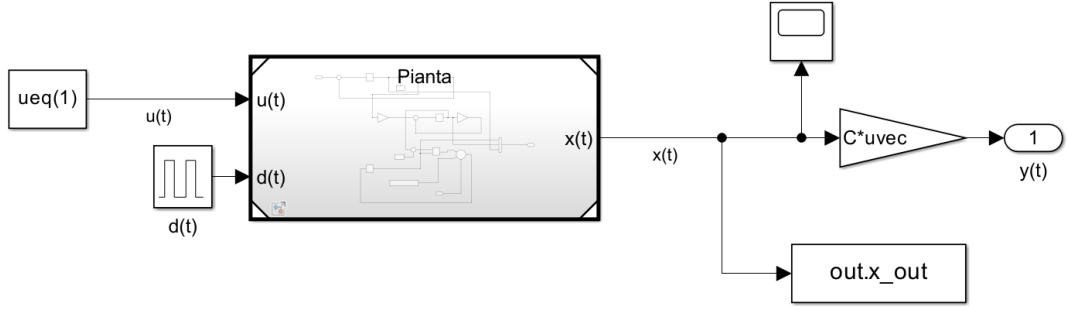


Figura 9: Open-loop system using Simulink.

The designed controller performance is evaluated through a simulation of 500(min). As for the inputs, a realistic operating scenario has been considered: the meal intake is represented as a pulse disturbance on $d(t)$, while the control input $u(t)$, which represent the insulin pump rate, is generated by the controller (we do not manually assign a predefined value). This approach will allow us to observe how the system reacts to a sudden glycaemic disturbance and how quickly it will restores to the equilibrium.

In particular $d(t)$ has been defined as:

$$d(t) = \begin{cases} 10, & \text{when } 10 \leq t \leq 40 \\ 0, & \text{otherwise} \end{cases} \quad (36)$$

The following figures show a comparison between the responses of the open loop system and the closed loop system, respectively, under identical inputs.

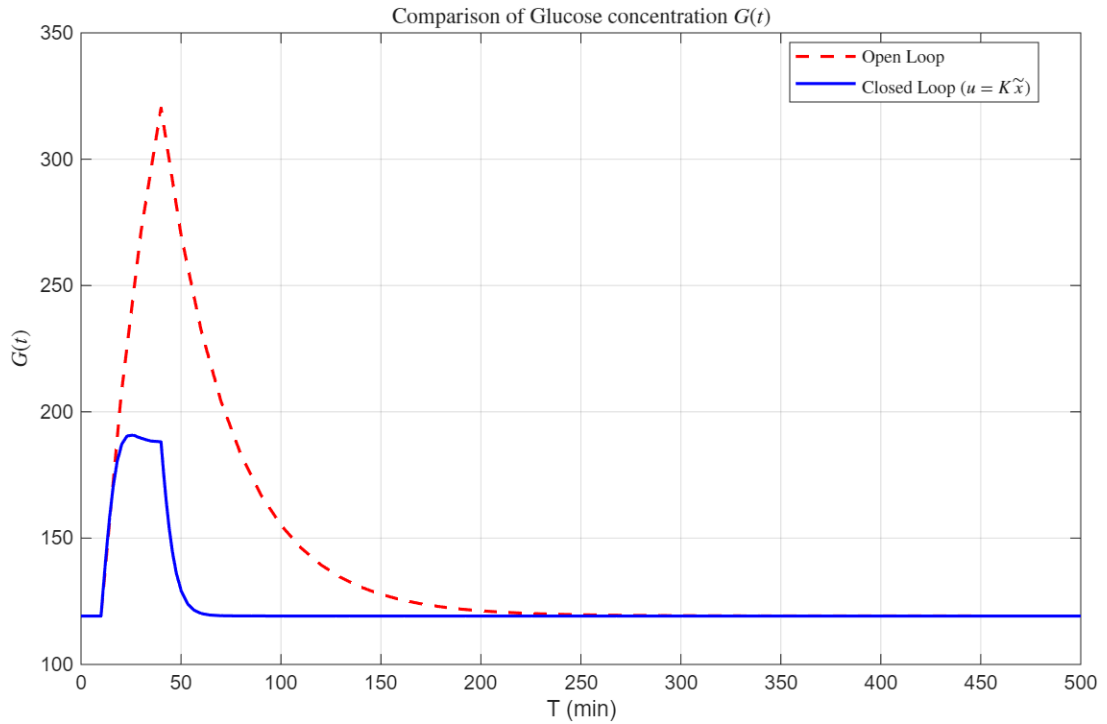


Figura 10: Plot comparison of $G(t)$.

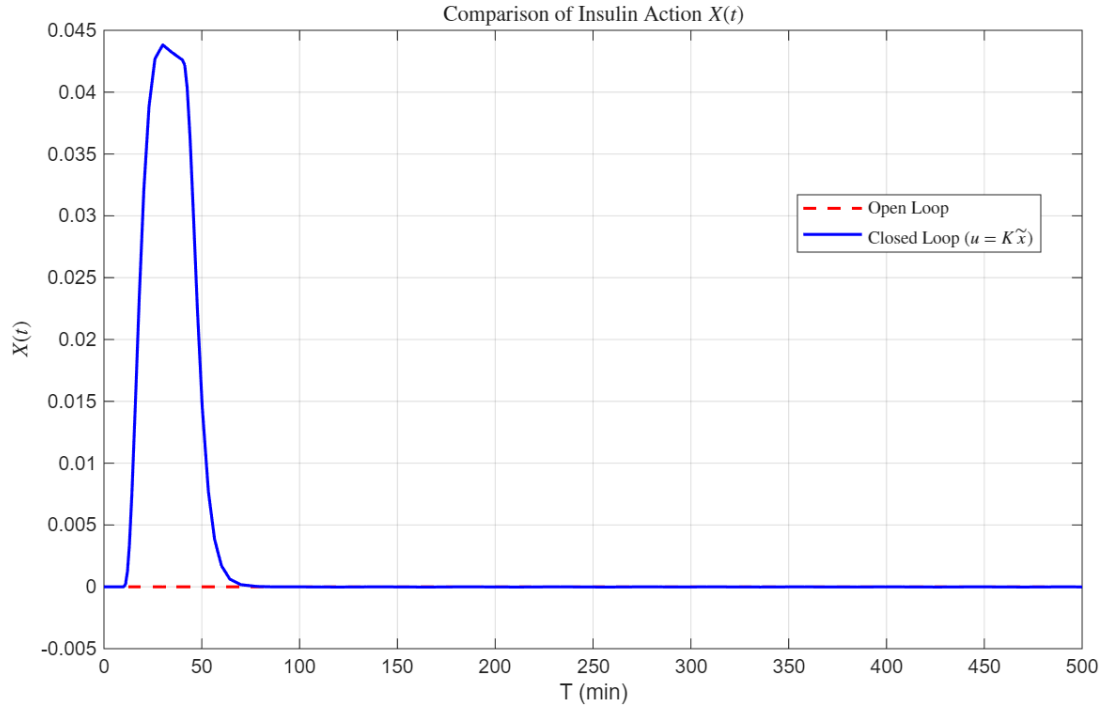


Figura 11: Plot comparison of $X(t)$.

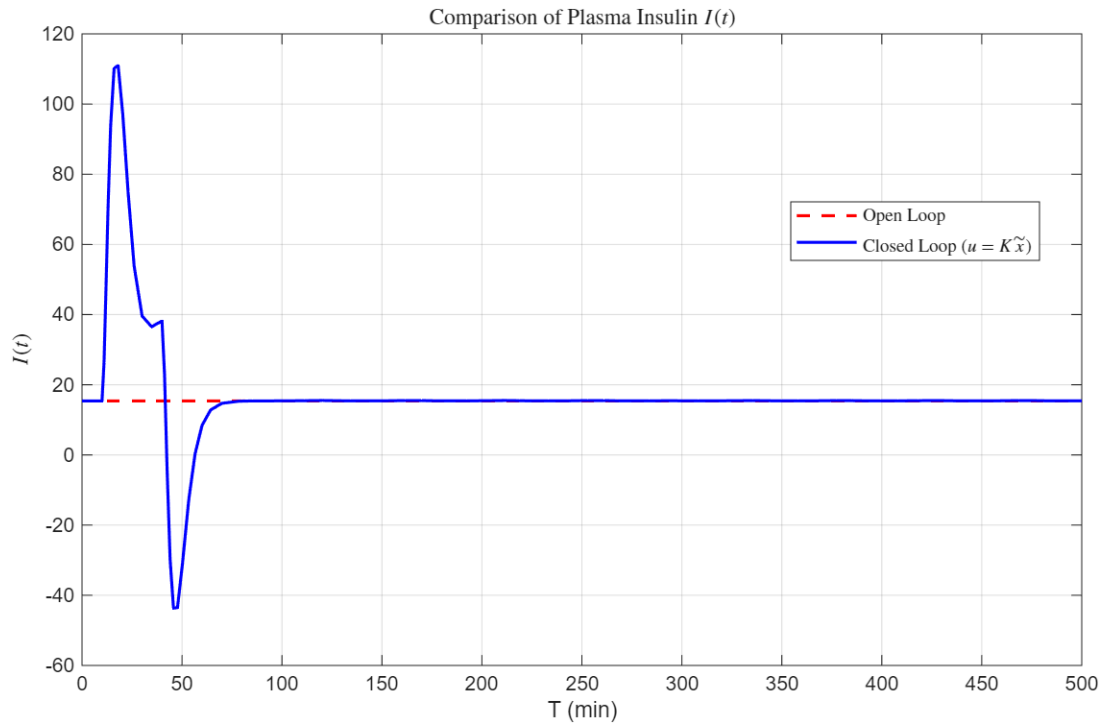


Figure 12: Plot comparison of $I(t)$.

The open-loop response represent the body dynamics in the absence of an artificial pancreas, while the closed-loop response shows how the insulin pump driven by a simple control law, such as the one implemented, can safely regulate the blood glucose levels and assist the patient. The first plot, Figure 10, is the most important: it shows how, after the meal disturbance, the open-loop response leads to a life-risking situation, in this case a hyperglycaemic condition, in contrast the closed-loop controller significantly reduces the glucose level and accelerates the re-

covery toward the equilibrium. As for the plots in Figure 11 and Figure 12, the open-loop system exhibits no variation in insulin levels in response to the rising of glucose. This accurately reflects the physical condition of a Type 1 diabetic patient, who lacks endogenous insulin production. Meanwhile, in the closed-loop system both the insulin action and plasma insulin concentration dynamically respond to counteract the rising glucose concentration.

Another important observation concerns the choice of the closed-loop poles. In this case, the poles chosen are about seven times faster than the open-loop eigenvalues of A . This means that the blood glucose level would theoretically reach the equilibrium much faster. However, this aggressive control action brings a risk of hypoglycaemia (due to the injection of an excessive amount of insulin). For this reason, a better choice would be given by:

```
1 %Choose closed-loop poles
2 poles_SSF = [-0.05, -0.06, -0.1];
```

Listing 20: Static State Feedback design using MATLAB.

In this case, the poles chosen are about two times faster than the open-loop ones. Even though it generates a slower transient response compared to a more aggressive tuning, it ensures that the control action remains within real limits, making it safer for the patient.

5.3 Asymptotic observer (Luenberger)

In practice, it's not always possible to directly measure all the states. For example, considering a real implementation of an Artificial Pancreas, we can observe that, while the plasma glucose concentration $G(t)$ can be continuously monitored with an appropriate sensor (like a CGM sensor), the insulin action $X(t)$ and the plasma insulin concentration $I(t)$ cannot be measured continuously in real time.

To overcome this limitation, we can implement an auxiliary dynamical system known as Asymptotic Observer. Based on available information of the system (plasma glucose concentration $G(t)$ and input $u(t)$), the observer estimates the remaining unknown states (the insulin action $X(t)$ and the plasma insulin concentration $I(t)$). Another advantage of this solution is that it reduces the number of required physical sensors.

Let $\hat{\tilde{x}}(t)$ denote the estimation of the state vector $\tilde{x}(t)$. A Luenberger asymptotic observer is defined as:

$$\begin{cases} \dot{\hat{\tilde{x}}}(t) = A\hat{\tilde{x}}(t) + B\tilde{u}(t) + L(\hat{\tilde{y}}(t) - \tilde{y}(t)) \\ \hat{\tilde{y}}(t) = C\hat{\tilde{x}}(t) + D\tilde{u}(t) \\ \hat{\tilde{x}}(0) = \hat{\tilde{x}}_0 \end{cases} \quad (37)$$

where $L \in \mathbb{R}^{n \times p}$ is the observer gain matrix. Meanwhile the estimation error of the state is defined as:

$$e(t) = \tilde{x}(t) - \hat{\tilde{x}}(t) \quad (38)$$

In particular, the observer in order to be asymptotic it must yield:

$$\lim_{t \rightarrow \infty} e(t) = 0 \quad (39)$$

Knowing that $D = 0$, we can write:

$$\tilde{y}(t) - \hat{\tilde{y}}(t) = C\tilde{x}(t) - C\hat{\tilde{x}}(t) = C(\tilde{x}(t) - \hat{\tilde{x}}(t)) = Ce(t) \quad (40)$$

This indicates that the output estimation error can be used as an indirect measure of the state estimation error, since $e(t)$ is unknown.

More interesting is the analysis of the error dynamics:

$$\begin{aligned}
\dot{e}(t) &= \dot{\hat{x}}(t) - \dot{\tilde{x}}(t) \\
&= A\tilde{x}(t) + B\tilde{u}(t) - A\hat{x}(t) - B\tilde{u}(t) - L[\hat{y}(t) - \tilde{y}(t)] \\
&= A[\tilde{x}(t) - \hat{x}(t)] - LC[\hat{x}(t) - \tilde{x}(t)] \\
&= (A + LC)e(t)
\end{aligned} \tag{41}$$

with $e(0) = \tilde{x}(0) - \hat{x}(0)$. Notice that the variation of the error depends on the design of matrix L . Furthermore, the observer is asymptotic if L is designed such that the eigenvalues of $(A + LC)$ are asymptotically stable.

As we did during the controller design, here, we can apply the eigenvalue allocation theorem. In particular, there will exist a matrix L such that the characteristic polynomial $\det(\lambda I - (A + LC)) = (\lambda - \bar{\lambda}_1)(\lambda - \bar{\lambda}_2) \dots (\lambda - \bar{\lambda}_n)$ matches any arbitrary chosen set of stable eigenvalues, if and only if the pair (A, C) is completely observable.

5.3.1 MATLAB and Simulink implementation

```

1 poles_obs = [-0.15, -0.20, -0.30];
2
3 Lt = place(A', C', poles_obs);
4 L=Lt';
5 AL = A-L*C
6 eig(AL)
7 BL = [B_u L]
8 CL = eye(size(A))
9 DL = zeros(size(A,1), 2)

```

Listing 21: Reachability using MATLAB.

where L is:

```

1 L =
2
3     0.5004
4    -0.0006
5    -0.2131

```

Listing 22: Reachability using MATLAB.

The matrices computed in the MATLAB code 22, are used to implement the observer in the Simulink State-Space block. The observer is initialized with the following initial condition: $x(0) = [0 \ 0 \ 0]^T$. It is important to notice that the observer design relies exclusively on B_u , the matrix related to the input $u(t)$. The non-manipulable disturbance $d(t)$ is entirely omitted from the observer's equations. This assumption makes sure to have a more realistic scenario to simulate: in reality, $d(t)$ which represent the meal intake, is usually hard to measure. Therefore the observer relies exclusively on the input $u(t)$ and the available state/output $G(t)$.

The Simulink implementation is the following:

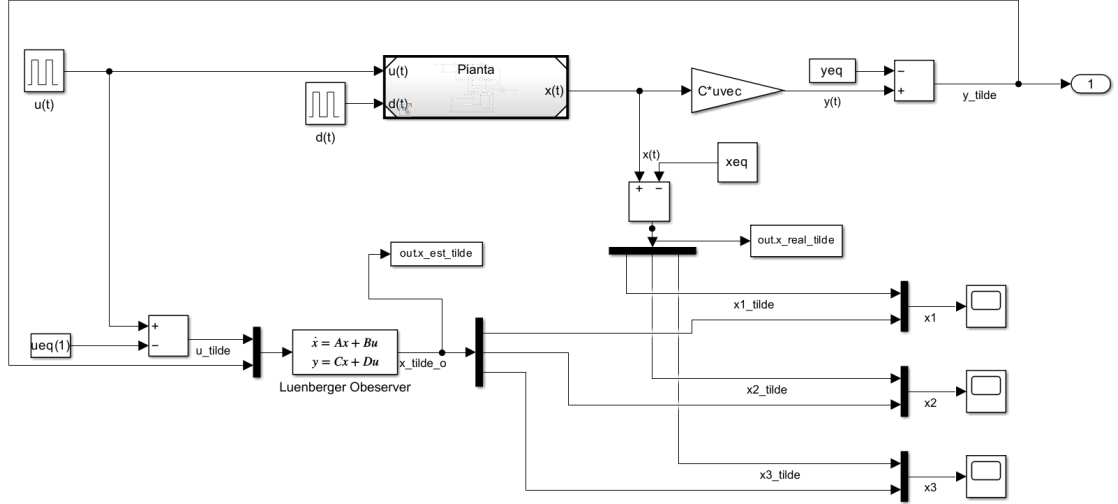


Figura 13: Luenberger asymptotic observer in the non-linear model using Simulink.

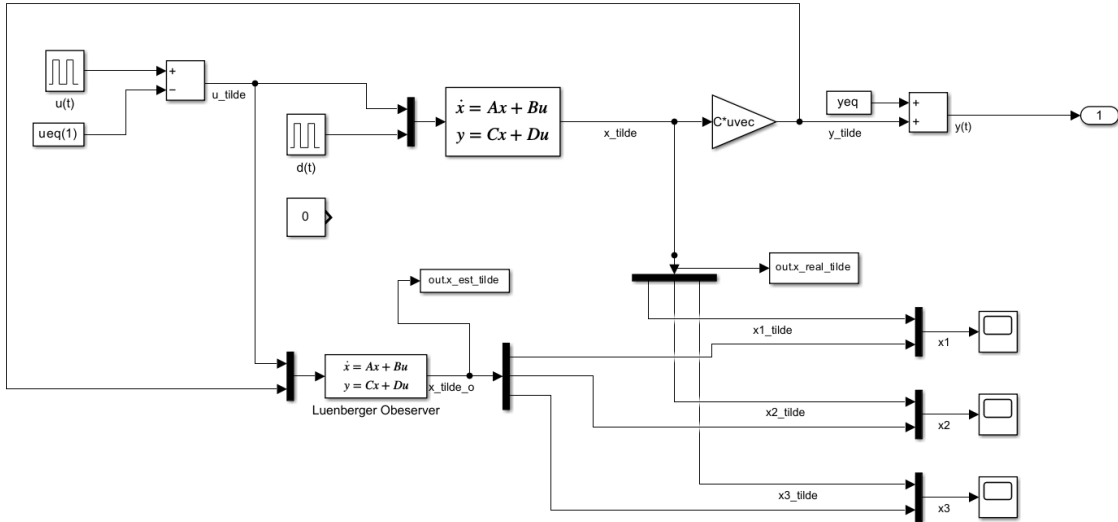


Figura 14: Luenberger asymptotic observer in the linear model using Simulink.

The designed observer is evaluated through a simulation of 500(min). The evaluation is structured into three distinct scenarios: two utilizing the linearized model and one applied to the non-linear model.

In all scenarios, the input $u(t)$ models a rapid insulin injection, defined as a pulse:

$$u(t) = \begin{cases} 15, & \text{when } 10 \leq t \leq 15 \\ 0, & \text{otherwise} \end{cases} \quad (42)$$

The input non-measurable and non-manipulable $d(t)$ is defined as a pulse representing a meal:

$$d(t) = \begin{cases} 10, & \text{when } 10 \leq t \leq 40 \\ 0, & \text{otherwise} \end{cases} \quad (43)$$

In the first scenario of the linear model, we use $u(t)$ as defined before, but $d(t) = 0$. This is an ideal case, that show the mathematical tracking ability of the observer in the absence of unmeasured inputs.

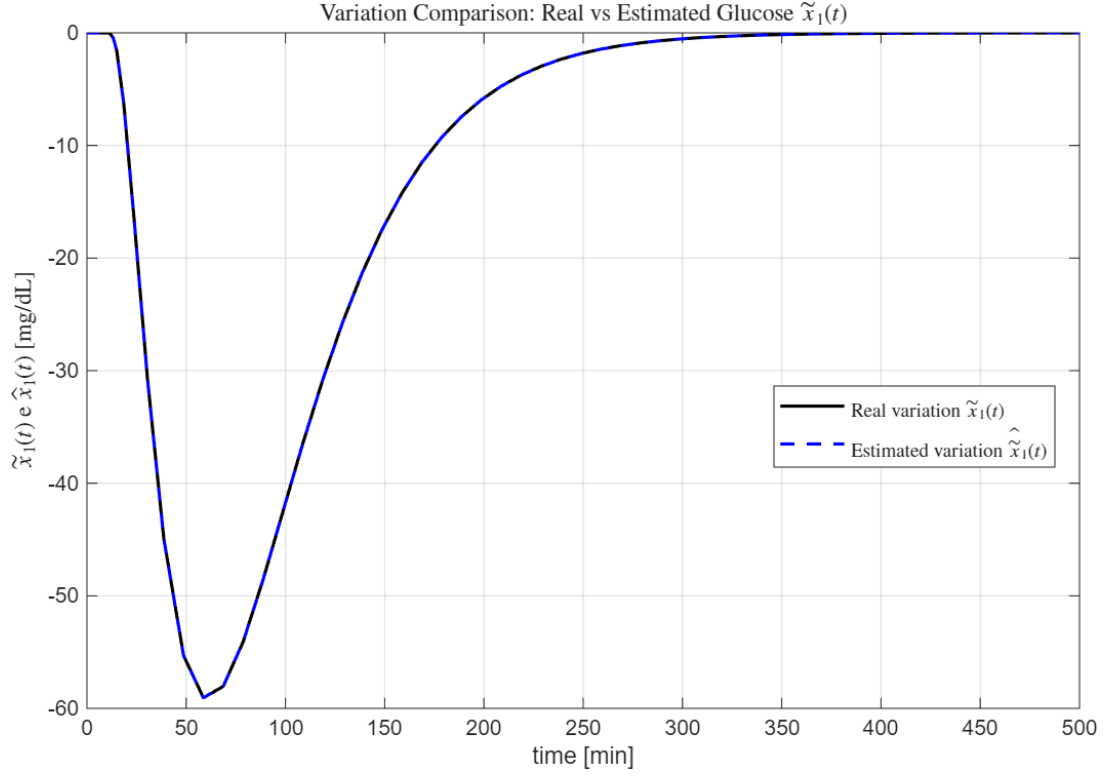


Figura 15: Comparison between $\tilde{x}_1(t)$ and $\hat{x}_1(t)$.

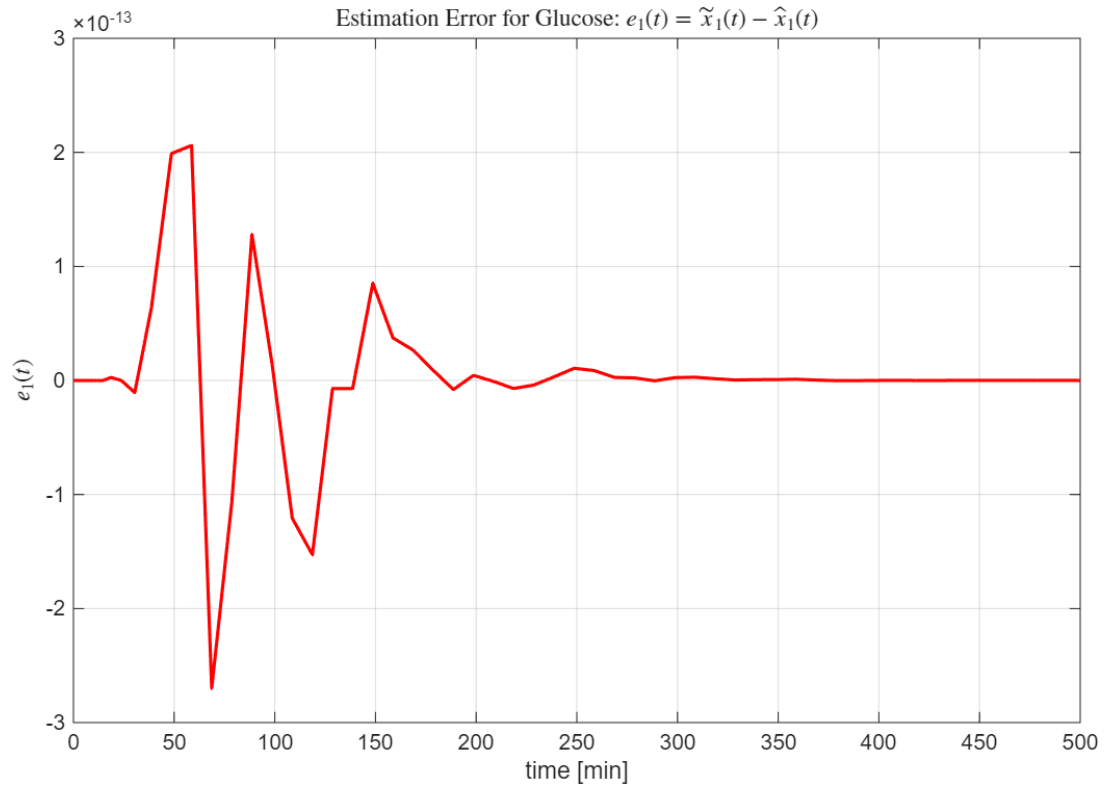


Figure 16: Error related to estimation of state $\tilde{x}_1(t)$.

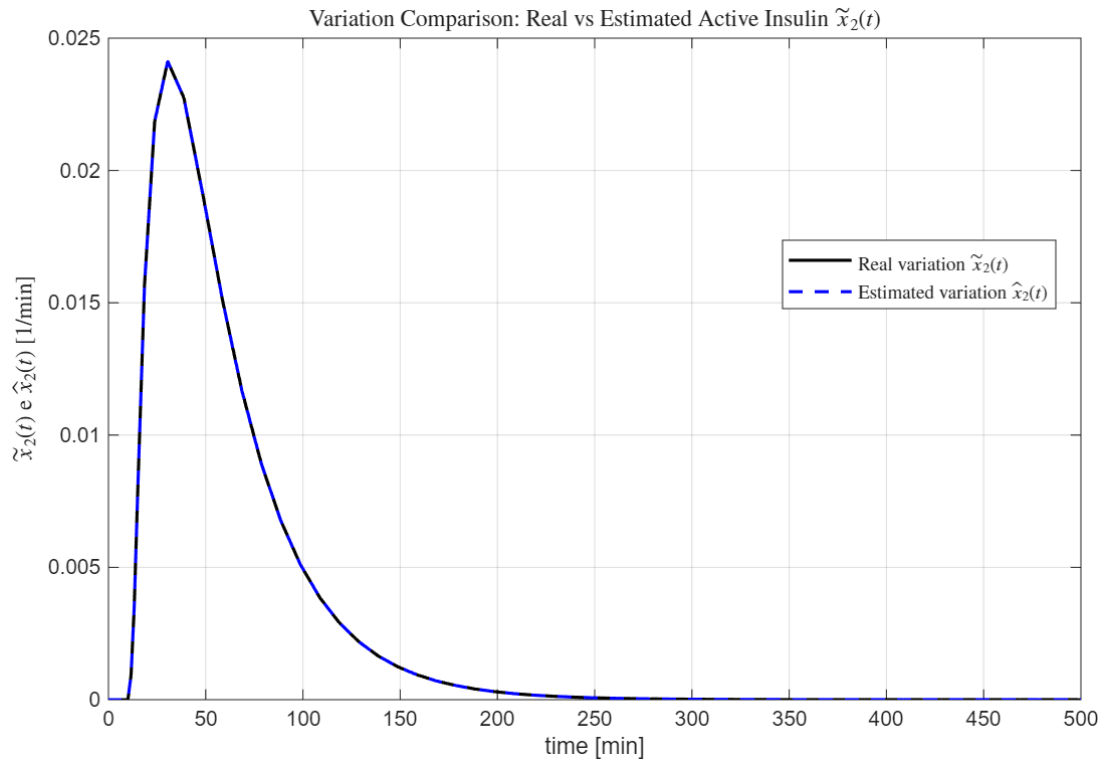


Figure 17: Comparison between $\tilde{x}_2(t)$ and $\hat{x}_2(t)$.

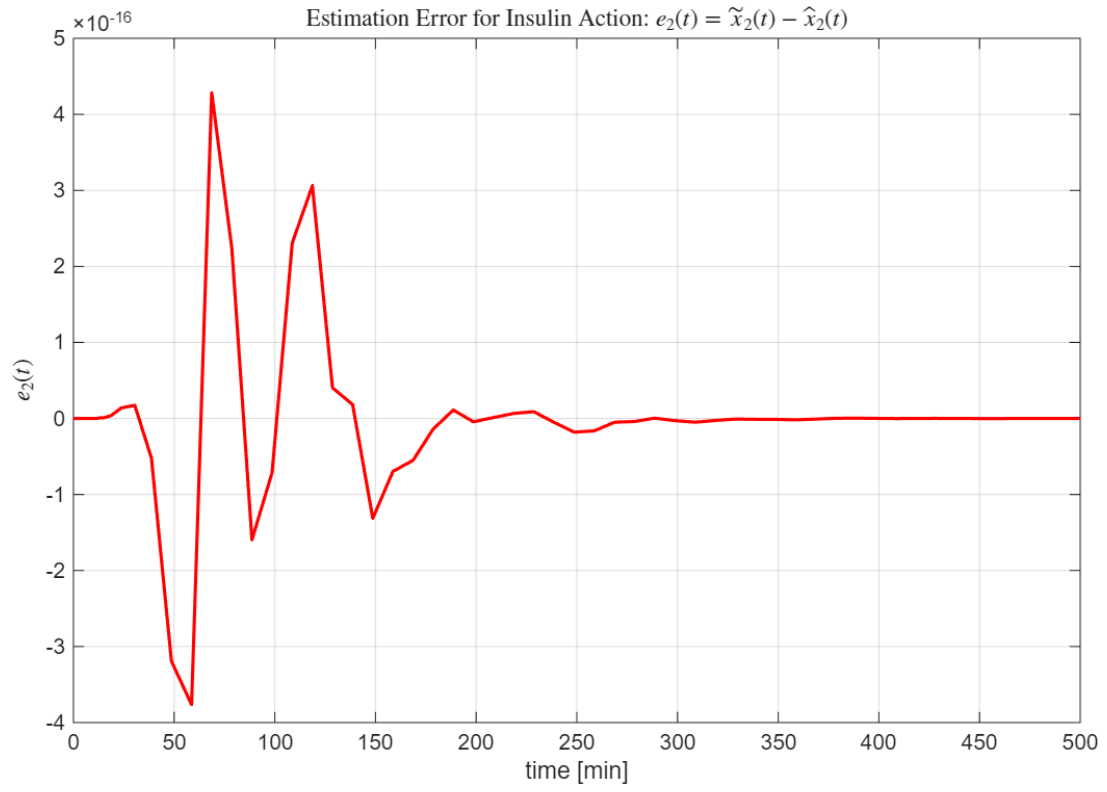


Figure 18: Error related to estimation of state $\tilde{x}_2(t)$.

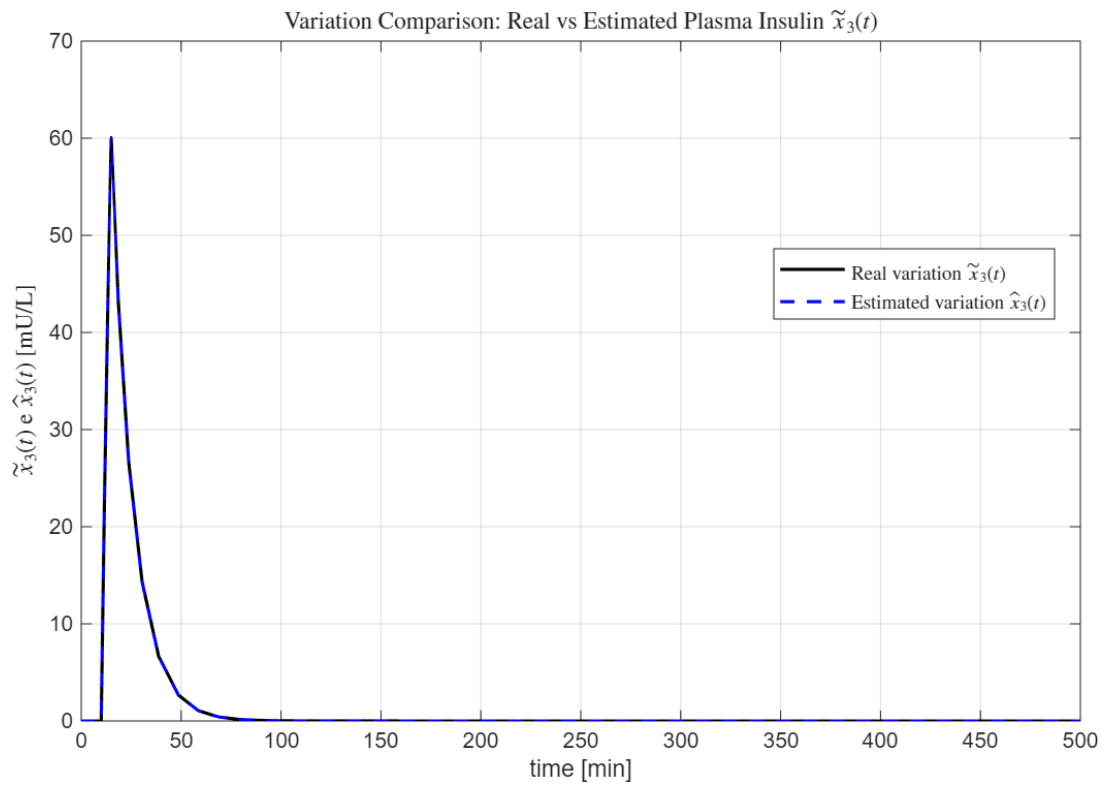


Figure 19: Comparison between $\tilde{x}_3(t)$ and $\hat{x}_3(t)$.

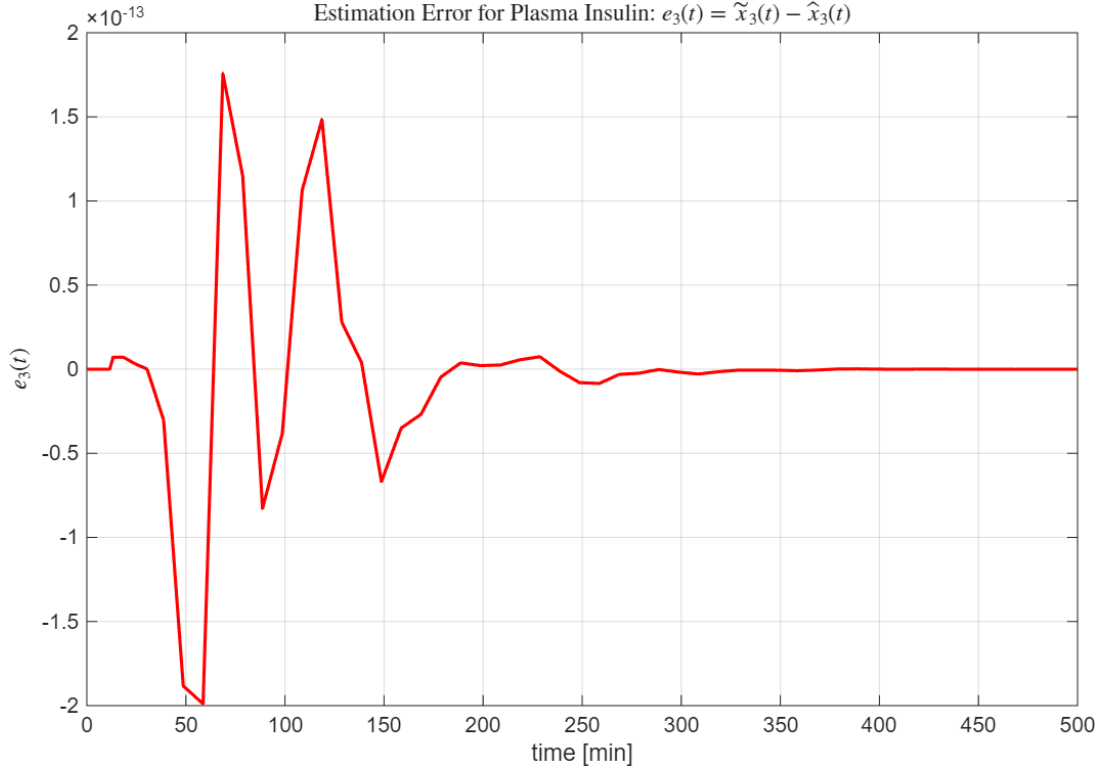


Figure 20: Error related to estimation of state $\tilde{x}_3(t)$.

The second scenario of the linear model uses both $u(t)$ and $d(t)$ as the pulse defined before. In this case we can see how the observer reacts to an unknown exogenous input.

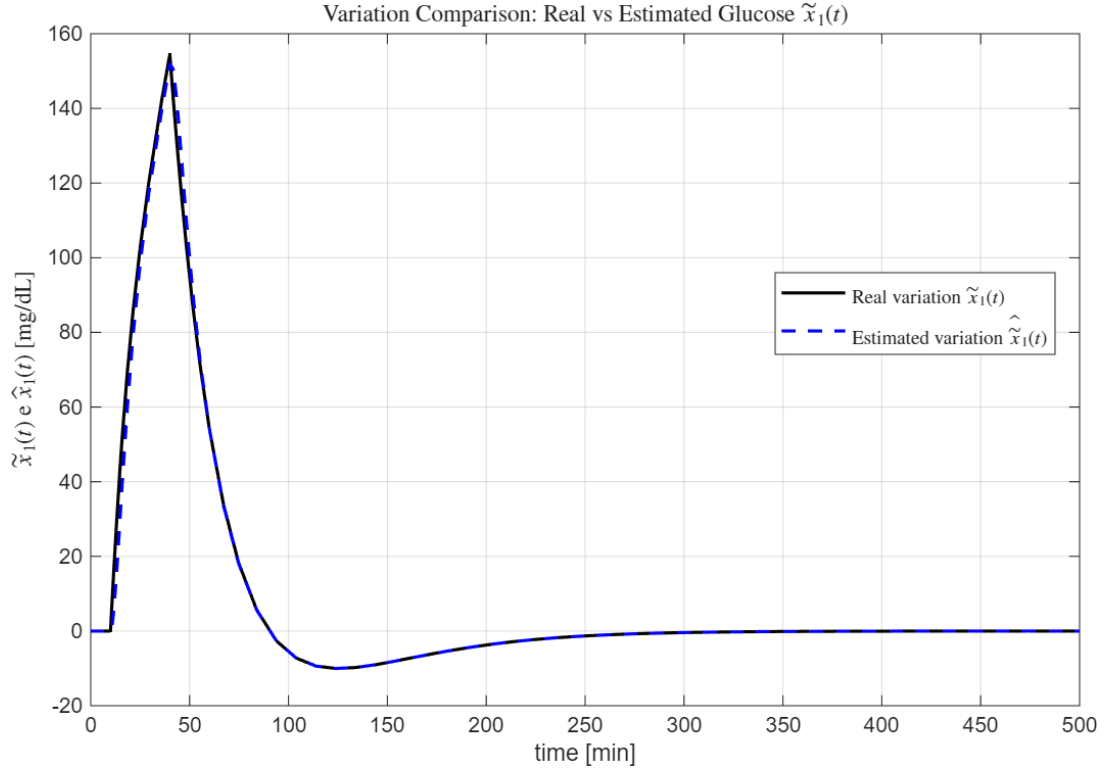


Figure 21: Comparison between $\tilde{x}_1(t)$ and $\hat{x}_1(t)$.

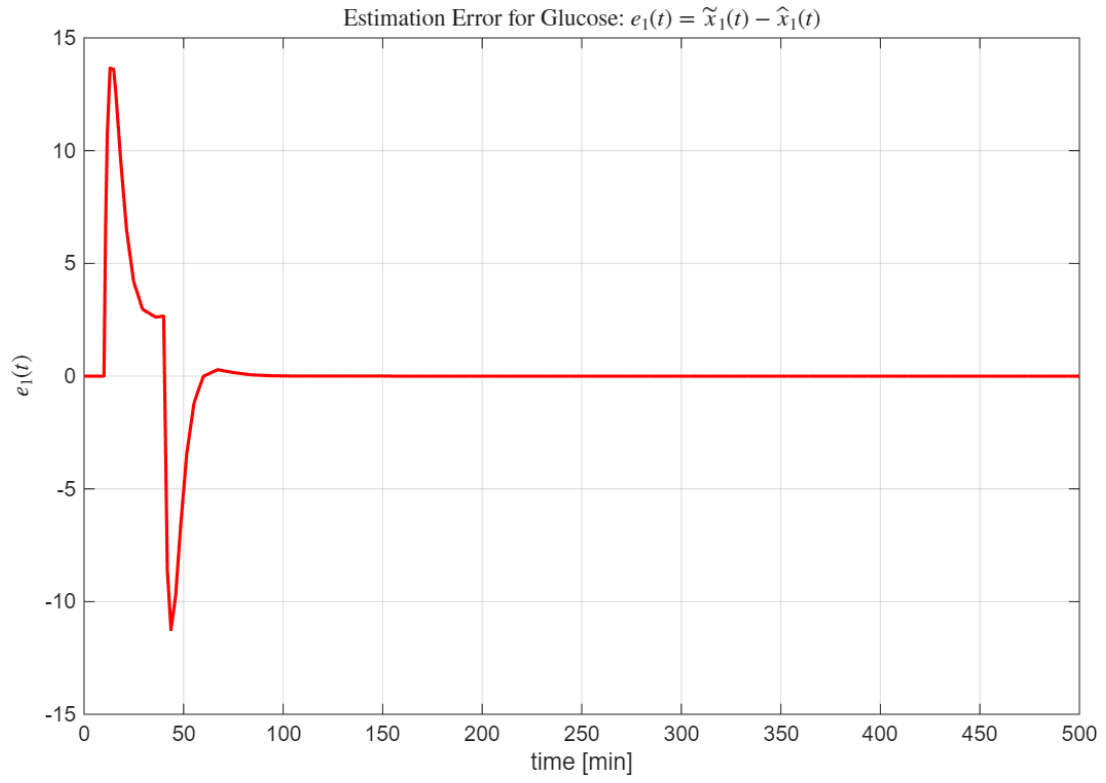


Figure 22: Error related to estimation of state $\tilde{x}_1(t)$.

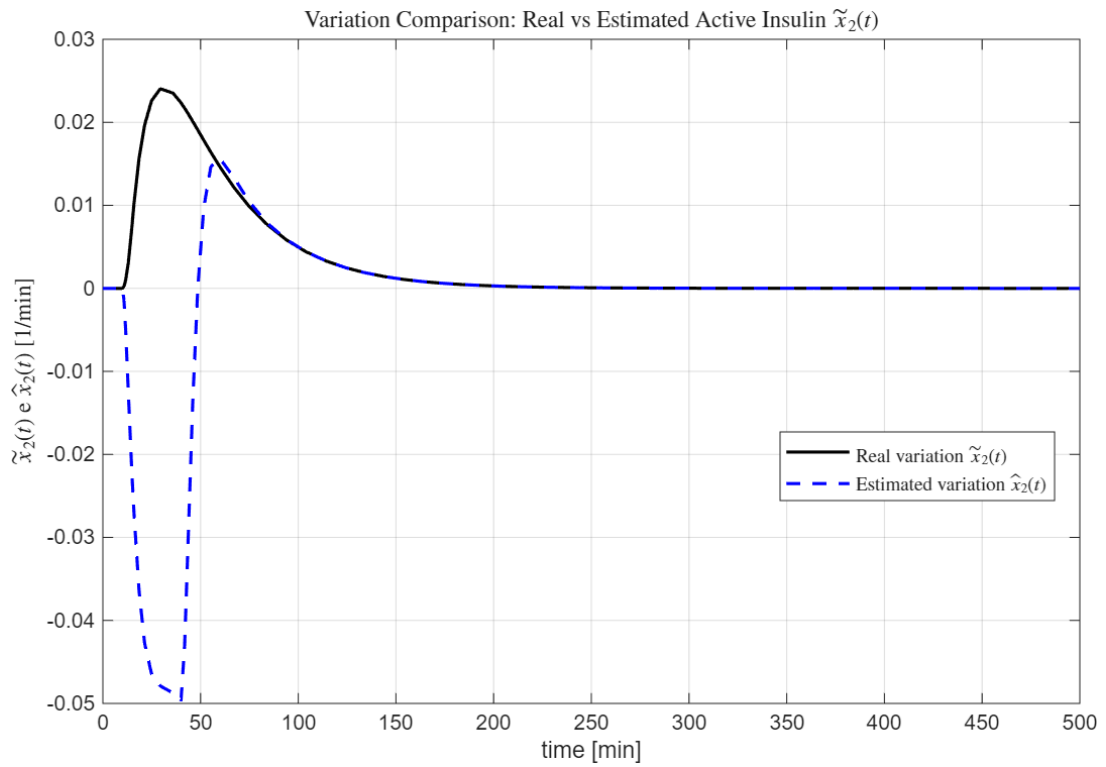


Figure 23: Comparison between $\tilde{x}_2(t)$ and $\hat{x}_2(t)$.

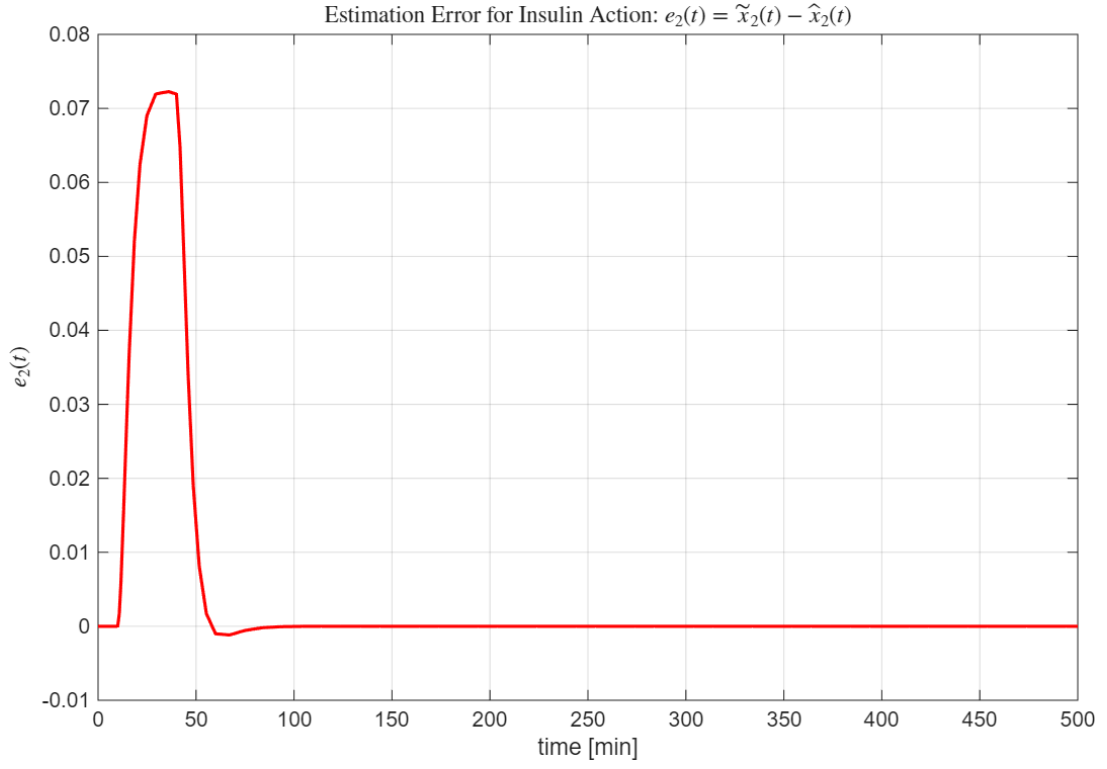


Figure 24: Error related to estimation of state $\tilde{x}_2(t)$.

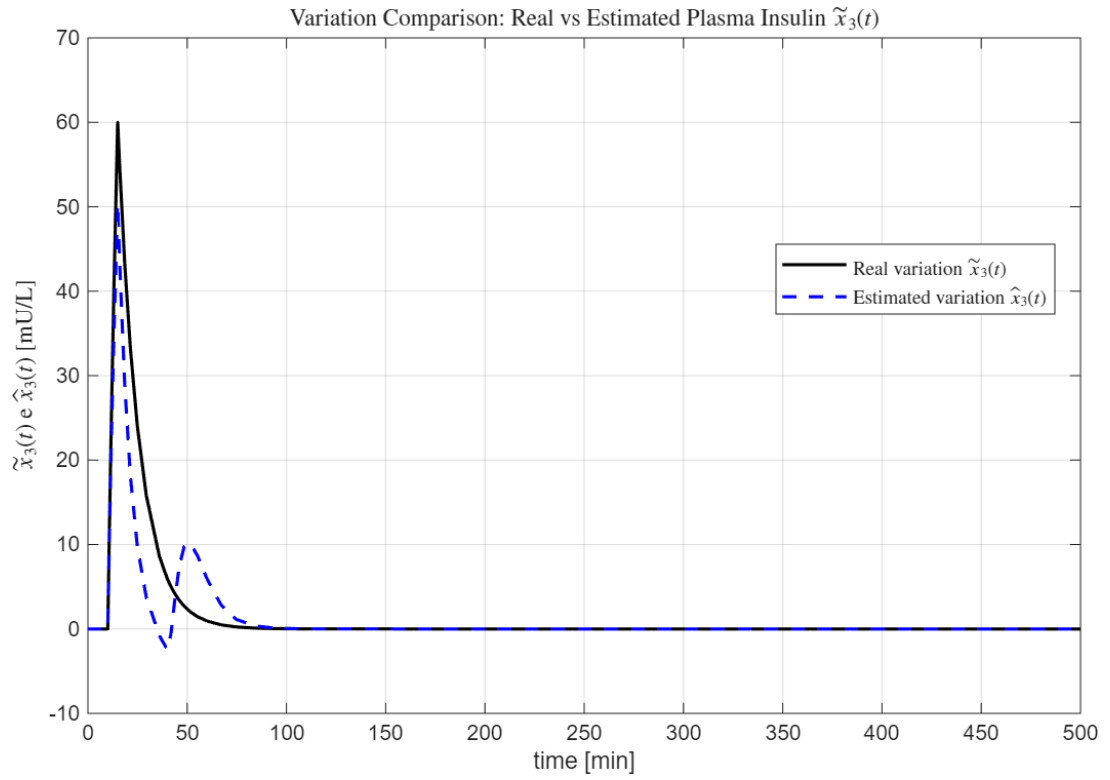


Figure 25: Comparison between $\tilde{x}_3(t)$ and $\hat{x}_3(t)$.

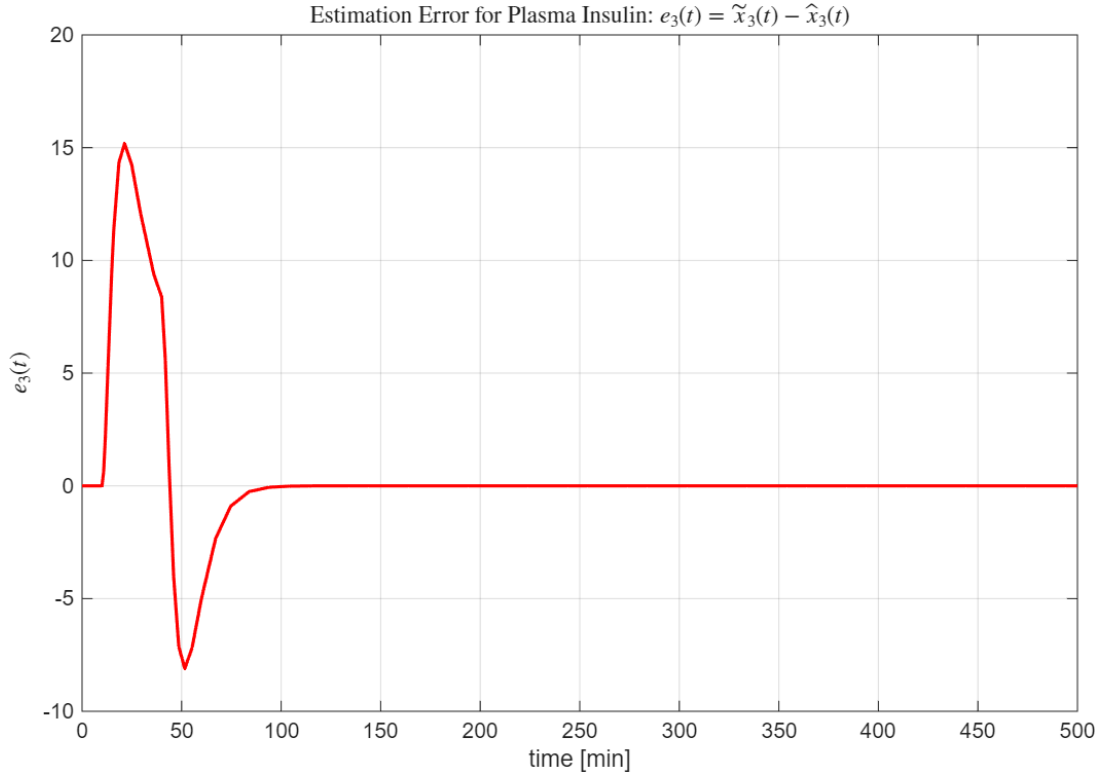


Figura 26: Error related to estimation of state $\tilde{x}_3(t)$.

As expected the estimation error in the second scenario is bigger than the first case, where the error was practically zero (infinitesimal order that can be ignored).

Lastly, the non-linear scenario utilizes the same inputs as the second scenario of the linear model. Here, we are interest in evaluating how the linear observer performs when applied to the non-linear model.

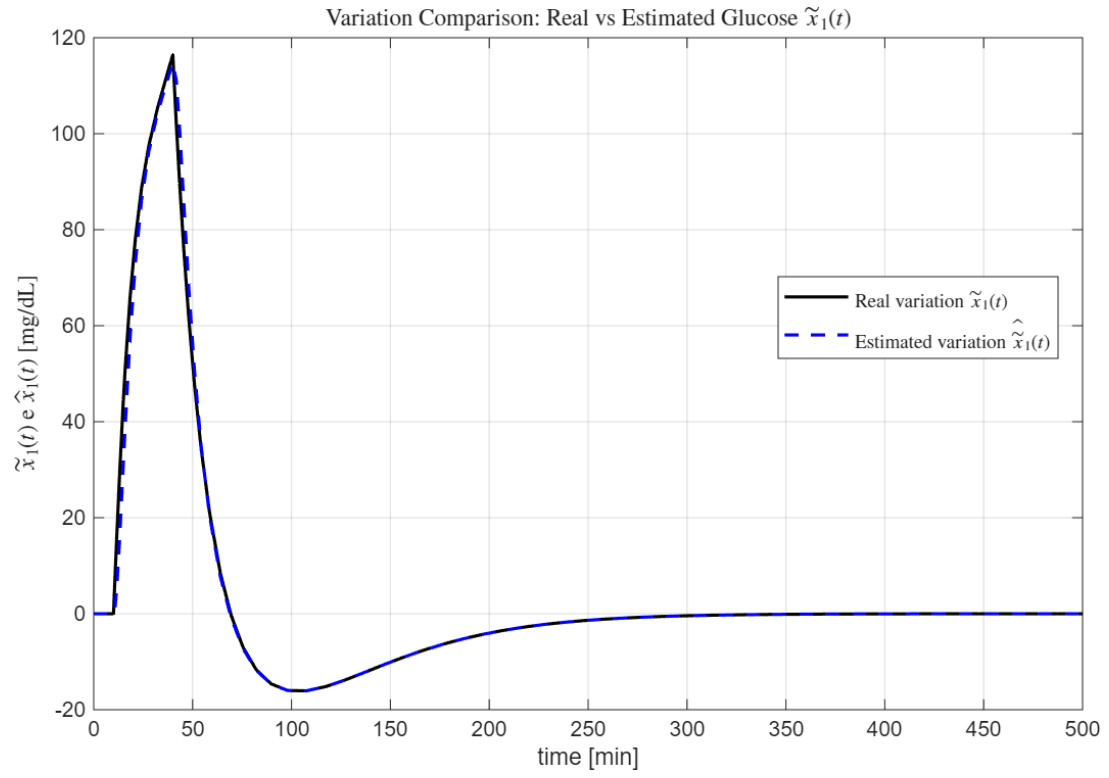


Figura 27: Comparison between $\tilde{x}_1(t)$ and $\hat{x}_1(t)$.

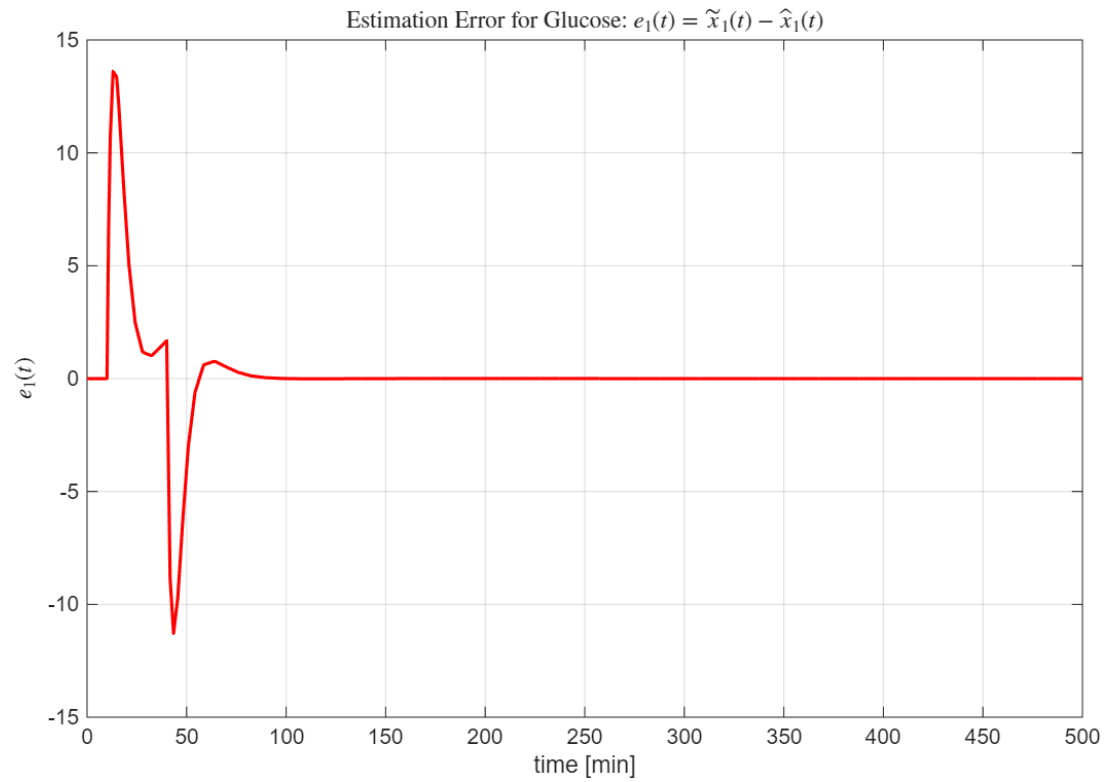


Figura 28: Error related to estimation of state $\tilde{x}_1(t)$.

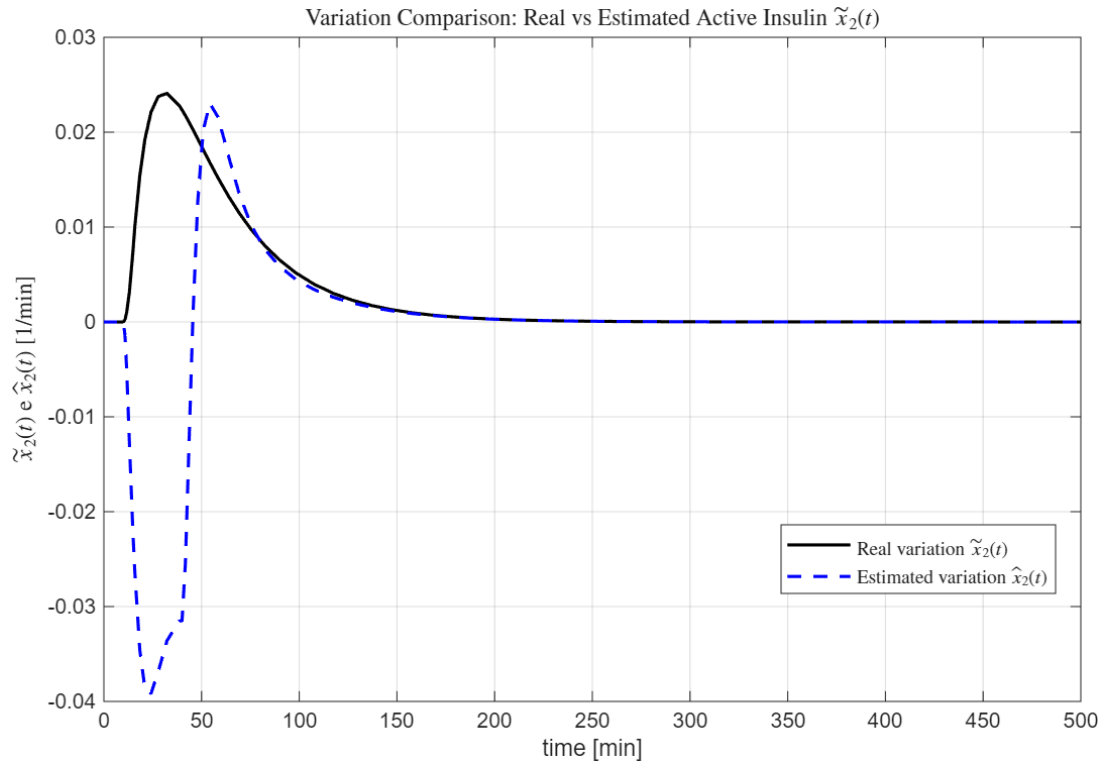


Figura 29: Comparison between $\tilde{x}_2(t)$ and $\hat{x}_2(t)$.

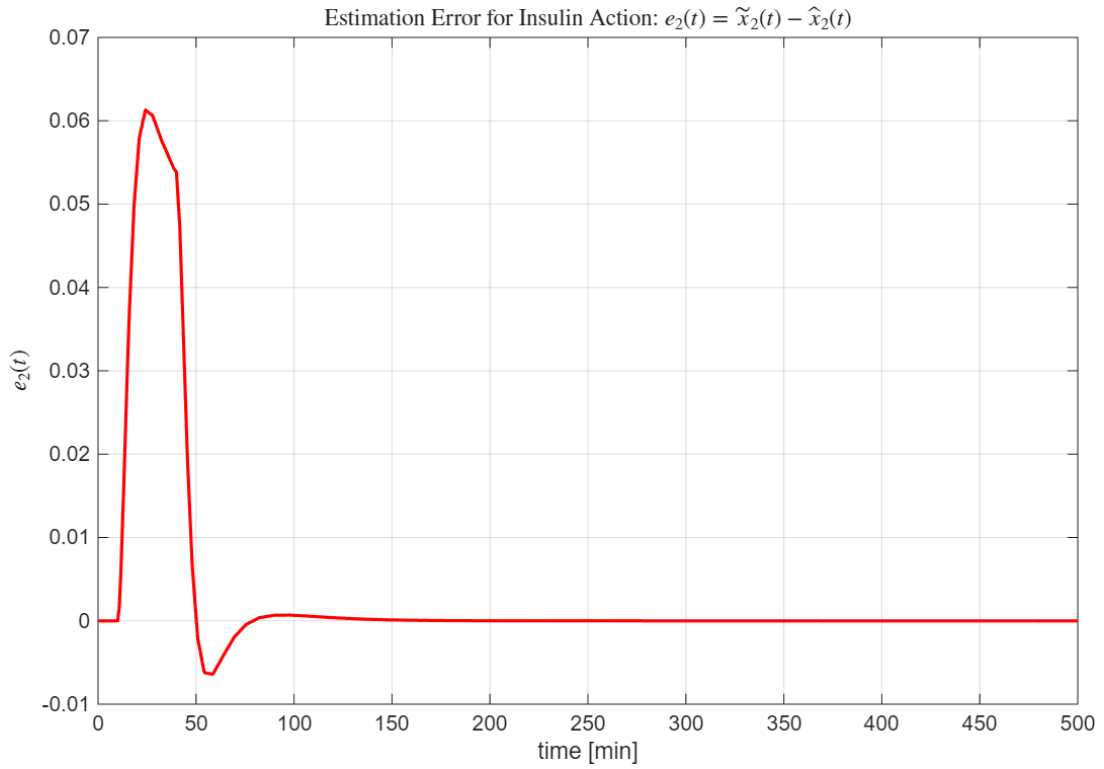


Figura 30: Error related to estimation of state $\tilde{x}_2(t)$.

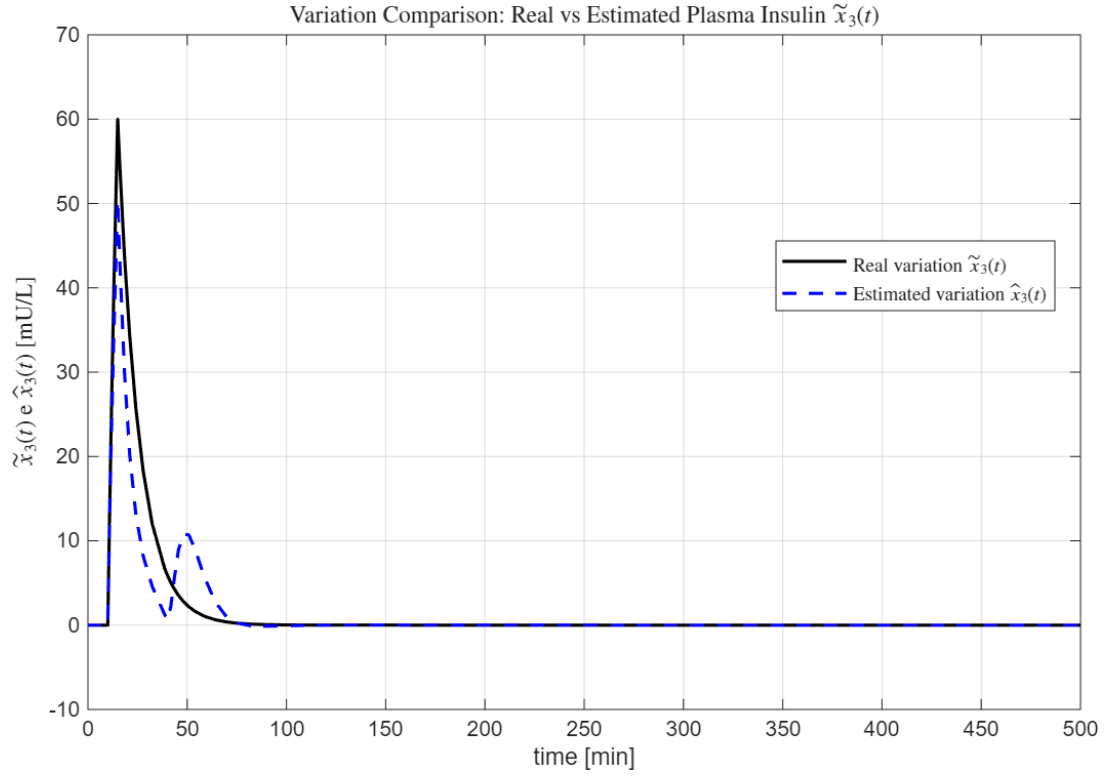


Figura 31: Comparison between $\tilde{x}_3(t)$ and $\hat{x}_3(t)$.

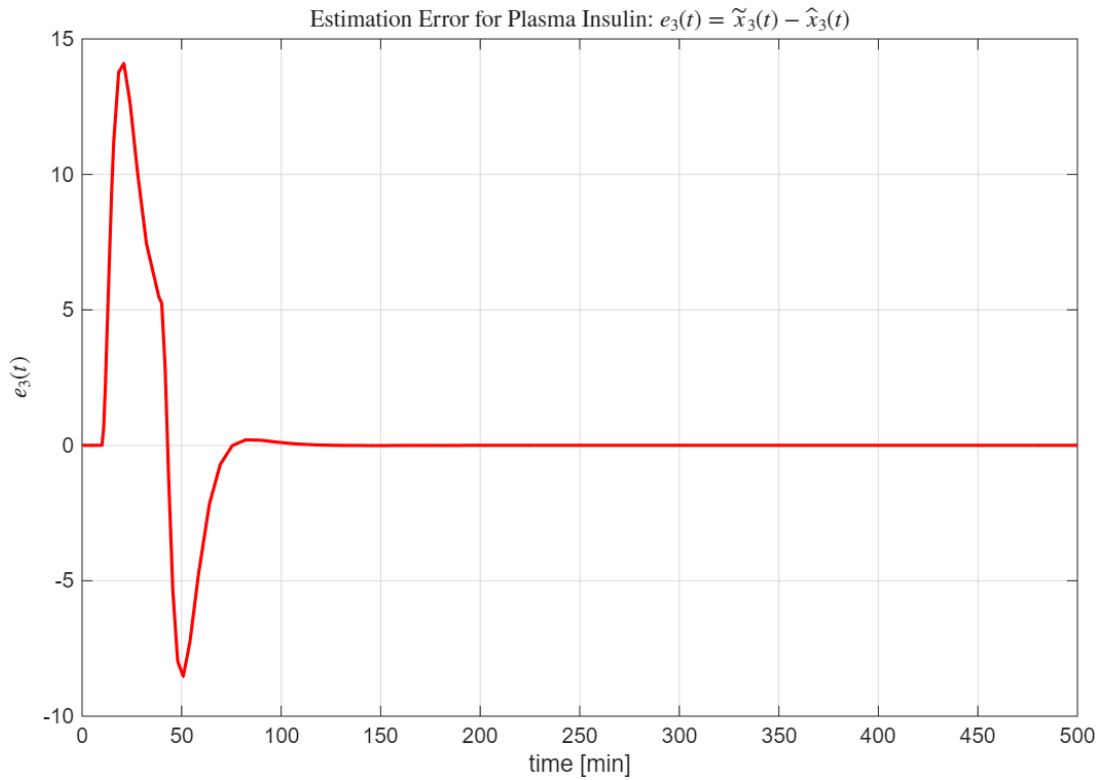


Figura 32: Error related to estimation of state $\tilde{x}_3(t)$.

To conclude, the first scenario represents the ideal case. Here, we see the mathematical accuracy of the design: in absence of $d(t)$, the observer perfectly tracks the system's states (the estimation

error is negligible). In the second scenario, the observe is subjected to the meal $d(t)$. Since it lacks direct information about this input, it try to compensates the sudden rise of blood glucose by adjusting the internal state estimates. However, once the meal dissipates, the estimation error successfully converges to zero. Lastly, in the third scenario we were interested into evaluating the linear observer applied to our non-linear model in presence of unmeasured meal input. Despite the initial estimation deviations caused by the combination of unmeasured mean and mathematical mismatch between the models, the observer eventually has an estimation error that asymptotically converge to zero.

5.4 Dynamic Compensator

As we said before, in section 5.2, the static state feedback cannot be directly implemented due to the presence of unmeasurable states ($X(t)$ and $I(t)$). The objective of a Dynamic Compensator is to utilize the state estimation provided by the asymptotic observer to implement the static state feedback.

Designing this regulator is possible as long as the system is stabilizable and detectable (note that these are the minimum conditions). Since our system is completely reachable and completely observable, we can design a dynamic compensator. Furthermore, we can combine the previous designed control law and asymptotic observer to obtain the compensator, thanks to the Separation Principle.

5.4.1 MATLAB and Simulink implementation

We apply the dynamic compensator to both the linearized and non-linearized systems in Simulink:

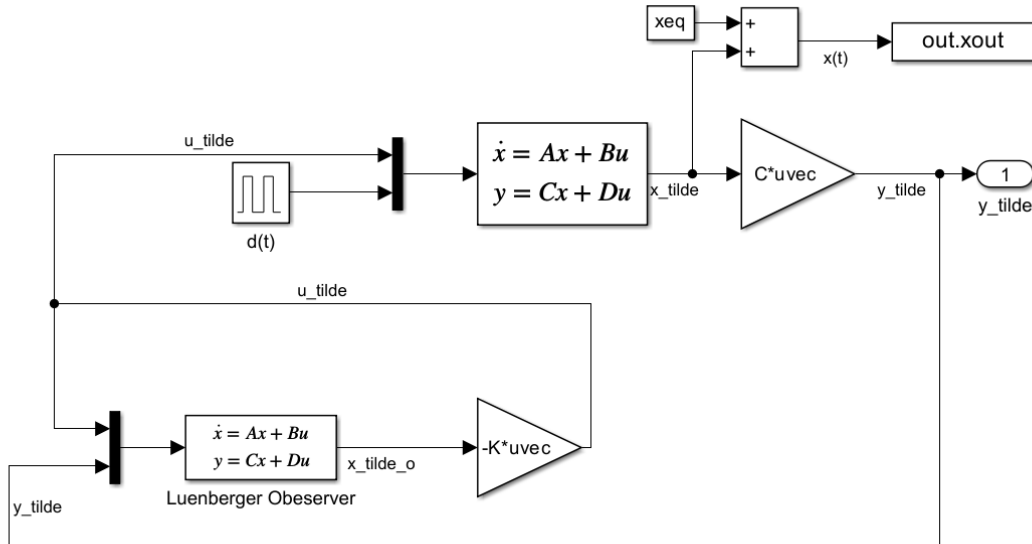


Figura 33: Dynamic compensator applied to the linear system.

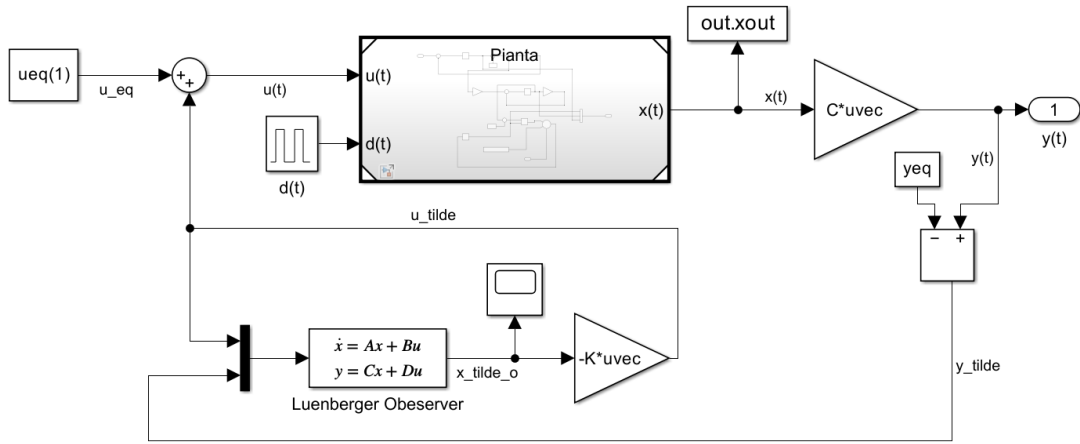


Figura 34: Dynamic compensator applied to the non-linear system.

Then we plot the responses obtained from a 500(min) simulation:

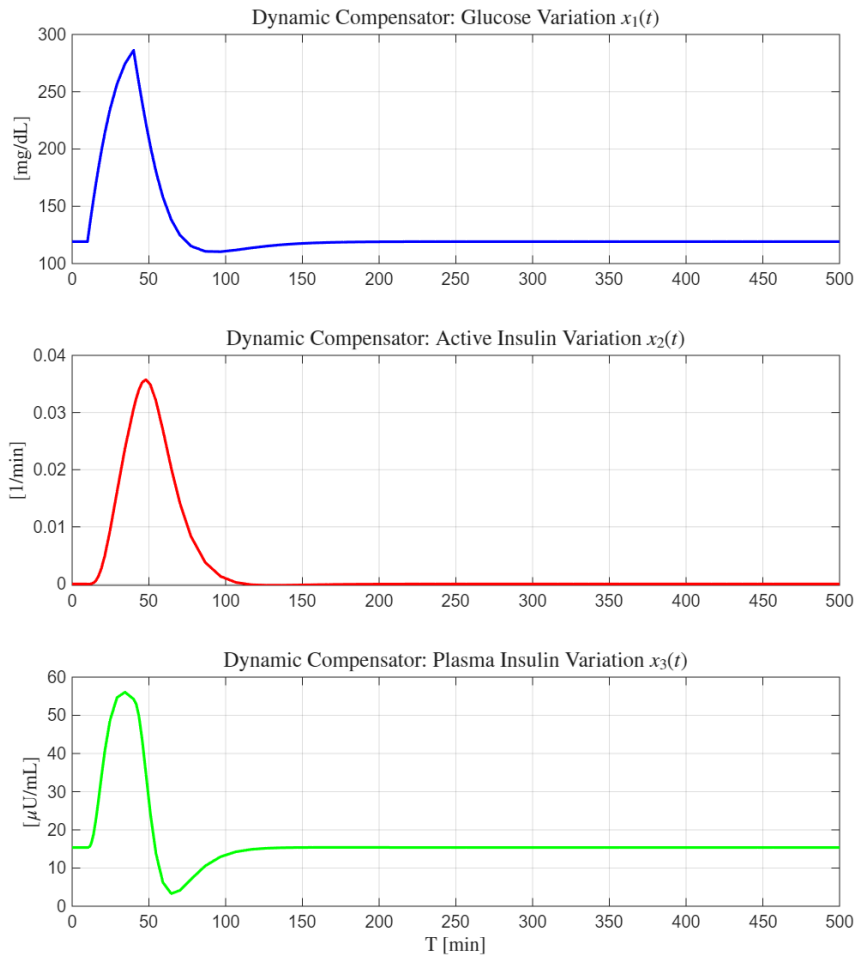


Figura 35: Response of the linear system with dynamic compensator.

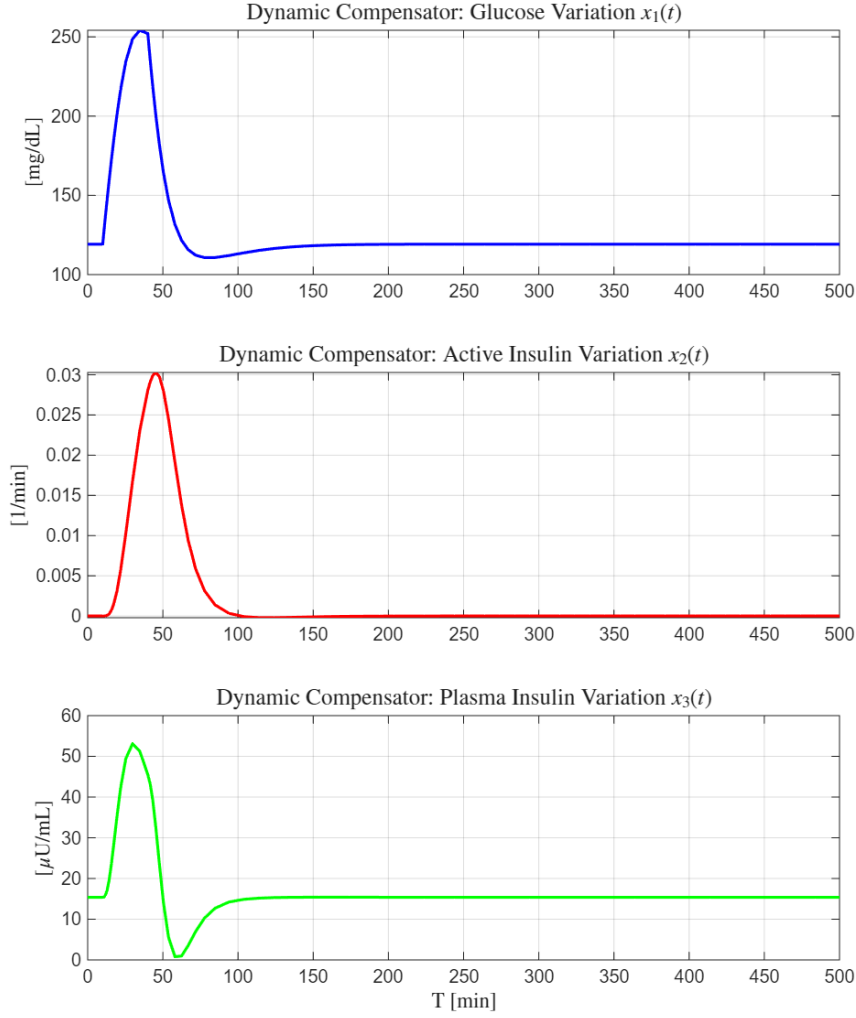


Figura 36: Response of the non-linear system with dynamic compensator.

As we can see, the dynamic compensator successfully manages the meal disturbance in both the linear and non-linear systems, bringing the blood glucose back to its basal level, without inducing dangerous hypoglycaemia. In particular, the closed-loop system presents significantly faster transient dynamics compared to the open-loop response. This confirms that the compensator successfully accelerates the system's regulation while maintaining the patient within a safe and stable glycemic range.

6 Conclusions

The objective of this project has been the analysis and regulation of an Artificial Pancreas for a Type 1 Diabetic patient. Starting from the non-linear equations of Bergman’s minimal model, the system was linearized around a basal equilibrium point. This allowed for the analysis of the system and its structural properties: the stability but slow dynamics of the systems showed the critical aspect in artificial pancreas design.

This limitation of the open-loop system, coupled with the vulnerability of the system to exogenous meal disturbances, justified the need for an active control strategy. Specifically, a state-feedback controller along with a state observer, since only glucose is measurable, were designed to artificially speed up the system’s dynamics, effectively reject meal disturbances, and safely maintain the patient in a safe state.

Finally, the combination of the state feedback controller and the asymptotic observer produced the dynamic compensator, which addressed the problem related to the unmeasurable states while fulfilling the control objectives. Simulation results, demonstrate that the proposed control architecture effectively accelerates the system’s natural dynamics even under meal-induced disturbances. Ultimately, the compensator maintains the patient within a safe glycaemic range.

Riferimenti bibliografici

- [1] R. N. Bergman, *Origins and History of the Minimal Model of Glucose Regulation*, Front. Endocrinol., 11:583016, 2021. doi: 10.3389/fendo.2020.583016.
- [2] S. Nandi and T. Singh, *Global Sensitivity Analysis on the Bergman Minimal Model*, IFAC-PapersOnLine, vol. 53, no. 2, pp. 16112–16118, 2020.